

TIME-LOCK PUZZLES FOR STATELESS RATE-LIMITING IN TRUSTED EXECUTION ENVIRONMENTS

Protecting Private Set Intersection Against Brute-Force Attacks

MORITZ MANTEL

BSc Computer Science
ETH Zurich

System Security Group
Nicolas Dutly · Prof. Dr. Srdjan Čapkun

July 2026

ABSTRACT

Trusted Execution Environments (TEEs) protect code and data from privileged adversaries, but their interfaces remain a critical attack surface. We consider this problem in the context of Private Set Intersection (PSI), where a service holds a private database and exposes an intersection interface. Even though the TEE preserves confidentiality of the database itself, an adversary can still reconstruct its entire contents by exhaustively enumerating all possible inputs. For low-entropy sets, this is a realistic scenario and thus needs to be addressed.

The canonical defense is rate-limiting, but implementing it securely inside a TEE is non-trivial. Popular TEEs such as Intel SGX lack a trusted time source and are vulnerable to rollback and cloning attacks. This allows an untrusted, privileged host to bypass traditional counter-based rate-limiters. Existing mitigations address these attacks through additional hardware primitives, but they add complexity and grow the Trusted Computing Base (TCB) considerably.

We explore client-side computational puzzles as an alternative. By shifting the rate-limiting burden to the client, this design avoids relying on trusted time or persistent security-sensitive state entirely, nullifying the effect of state-continuity attacks. We present and evaluate an Intel SGX prototype implementation and the experimental results broadly support the theoretical guarantees. The evaluation also highlights the limitations of our proposed approach. The cost imposed on an adversary depends on hardware and implementation choices that are outside the rate-limiter’s control, making precise security guarantees difficult. For low-entropy sets, the cost enforced on legitimate clients may become impractical.

These tradeoffs suggest that client-side puzzles are an interesting alternative in specific deployment scenarios, in particular when the universe is large and legitimate clients query only a small fraction of it, but not a universal replacement for traditional rate-limiting.

ACKNOWLEDGMENTS

First and foremost, I would like to thank my supervisor, Nicolas Dutly, for his guidance and support throughout this project. His thoughtful feedback and insights were invaluable and I greatly enjoyed our discussions. Thank you for giving me the freedom to focus on what I found most compelling, but still guiding me to a complete thesis.

I am also grateful to Prof. Dr. Srdjan Čapkun for giving me the opportunity to work on this topic within the System Security Group at ETH Zurich.

Finally, I would like to thank my friends and family. Whether it was thought-provoking discussions, proof-reading the final drafts, listening to my monologues about my ideas and challenges, or preparing lunch while I was finishing up on a chapter, I greatly appreciated all your support!

Zurich, July 2026

Moritz Mantel

CONTENTS

1	Introduction	1
2	Background	4
2.1	Trusted Execution Environments	4
2.1.1	Motivating TEEs	4
2.1.2	Threat Model of TEEs	4
2.1.3	Security Guarantees of TEEs	5
2.1.4	How TEEs Work	5
2.1.5	Limitations of TEEs	6
2.2	Rate Limiting	9
2.3	Private Set Intersection	9
3	Strawman Solutions	11
3.1	Attempt 1: Pure Cryptography	11
3.1.1	Limitation	11
3.2	Attempt 2: Rate-Limiting	12
3.2.1	Security Analysis	13
3.2.2	Limitation	13
3.3	Attempt 3: Rate-Limiting + TPM	13
3.3.1	Limitation	14
3.4	Attempt 4: Rate-Limiting + TPM + MC	16
3.4.1	Limitation	16
3.5	Summary of Strawman Solutions	17
4	Rate-Limiting Alternatives	19
4.1	Computational Puzzles	19
4.1.1	Puzzle-Based Rate-Limiting Design	19
4.1.2	Work-Depth Model	20
4.2	Proof of Work	21
4.2.1	Quantitative Guarantee	21
4.3	Memory Hard Functions	21
4.3.1	Quantitative Guarantee	22
4.4	Sequential Functions	22
4.4.1	Verifiable Delay Functions	22
4.4.2	Time-Lock Puzzles	22
4.4.3	Quantitative Guarantee	23
4.5	Time Lock Puzzles for PSI	23
4.5.1	Stateless Design	23
4.5.2	Two-Phase Protocol Design	25
4.5.3	Security Guarantees	25
4.6	Summary of Alternative Rate-Limiting	25
5	Implementation of Time-Lock Puzzles	27
5.1	Implementation	27
5.1.1	Environment	27
5.1.2	Architecture	28

5.1.3	Design Choices	28
5.1.4	Configurations	29
5.2	Evaluation Setup	30
6	Evaluation	32
6.1	Measurements	32
6.1.1	Linear Scaling in T	32
6.1.2	Additional Measurements	33
6.1.3	Required Cycles	33
6.1.4	Comparing Wall-Clock Time	35
6.2	Security Analysis	36
6.2.1	Security Properties of PSI	36
6.2.2	Attacks	37
6.3	Adversary Instantiations	39
6.3.1	Adversary Models	39
6.3.2	Practical Examples of Private Sets	41
6.3.3	Summary of Adversarial Models and Examples	42
7	Discussion	44
7.1	Limitations	44
7.2	Future Work	45
7.3	Choosing a Rate-Limiting Primitive	46
7.4	Conclusion	47
A	Appendix	48
A.1	Source Code	48
A.1.1	Solver Implementations	48
A.2	Further Evaluation Figures	49
A.3	Use of Generative AI	51
	Bibliography	53
	Declaration of Originality	57

LIST OF FIGURES

Figure 1	Threat model and problem setting: A private set exposes an intersection interface to clients, including the host. Both clients and the host are untrusted. The host has a privileged position over the PSI service, providing resources, having control over scheduling and memory. .	1
Figure 2	Protocol to establish a secure channel between enclave and Trusted Platform Module (TPM) for trusted time retrieval. Note that the TPM's public key is hardware backed and certified by the manufacturer, allowing the enclave to detect a spoofed or substituted TPM.	14
Figure 3	Rollback attack: setting the enclave back to previous state by replaying stale state	15
Figure 4	Rollback Example: rolling back to reset the counter to zero	15
Figure 5	Cloning Attack: each clone gets handed a fraction of the requests	17
Figure 6	High-level design of rate-limiting using computational puzzles	20
Figure 7	Design of the Intel SGX TLP implementation. .	28
Figure 8	Fixing all other configurations, the time to solve a puzzle scales linear in T , while generating and verifying a puzzle stays constant.	32
Figure 9	The number of cycles measured in each iteration of a squaring loop for multiple T . The required cycles are largely independent of the iteration and T , allowing us to set a lowerbound.	33
Figure 10	The average cycles used per squaring iteration by the <i>OpenSSL-loop</i> implementation compared to <i>mbedtls</i> with different T	34
Figure 11	The total number of cycles required by <i>mbedtls</i> and <i>OpenSSL-loop</i> for increasing T : The <i>OpenSSL-loop</i> implementation is considerably more efficient, but both are linear in T	35
Figure 12	Comparison of the latency of different solvers on different hardware. While they are all linear in T , their efficiency varies drastically.	36

Figure 13	Comparison of constant time comparison and the built in mpi comparison, which leaks the length of the matching prefix.	38
Figure 14	Scaling of solve time with input size: with T and all other configurations fixed, linear scaling in input size has the expected linear consequence for the solver time.	49
Figure 15	Scaling of solve time with input size: with T and all other configurations fixed, the linear scaling does not influence generation or verification time.	50
Figure 16	Configuring <code>T_baseline_comp=1</code> , T is computed such that the expected number of cycles to solve the puzzle is at least the Expected Cycles . As a reference, 300'000 cycles per iteration was used.	50
Figure 17	Configuring <code>T_baseline_comp=1</code> and <code>T_private_set_comp=1</code> , T is computed such that the expected number of cycles to hit an element in $\mathcal{S}$ when randomly probing is at least Expected Cycles . As a reference, 300'000 cycles per iteration was used. The solve time scales linearly in $\frac{ \mathcal{S} }{ U }$. The fluctuations might be explained through rounding errors.	51
Figure 18	The latency of initialising the crypto primitives scales exponentially in the number of bits used for the modulus. The overhead of ecalls is in the order of a few milliseconds.	51

LIST OF TABLES

Table 1	TEEs and their available time source	7
Table 2	Security Guarantees of Private Set Intersection	10
Table 3	Summary of strawman attempts and the attacks they address.	18
Table 4	Definition of the domains of the work-depth model.	21
Table 5	Summary of computational puzzles and their guarantees.	26
Table 6	Implementation Environment	27
Table 7	Overview of configurations implemented.	29
Table 8	Hardware details and library versions used for the evaluation.	30

Table 9	Explanation of the measurement method for the terms used in the evaluation.	31
Table 10	Instantiations of adversarial models.	39
Table 11	Summary of private set examples and derived parameters.	42
Table 12	Overview of generative AI usage.	52

LIST OF CODE SNIPPETS

1	Solver implementation: <i>mbedtls</i>	48
2	Solver implementation: <i>OpenSSL-loop</i>	48
3	Solver implementation: <i>OpenSSL-exp</i>	49
4	Solver implementation: <i>python</i>	49

ACRONYMS

MC	Monotonic Counter
MHF	Memory Hard Function
PoW	Proof of Work
PSI	Private Set Intersection
TCB	Trusted Computing Base
TEE	Trusted Execution Environment
TLP	Time-Lock Puzzle
TPM	Trusted Platform Module
TSC	Time Stamp Counter
VDF	Verifiable Delay Function

INTRODUCTION

Consider a cybersecurity firm that performs forensic investigations on data breaches. Over time, it accumulates a database of compromised accounts. The firm exposes a public interface allowing clients to check whether their accounts appear in that database. The full set must remain private, however, as it would reveal sensitive information to a potential adversary. Clients therefore only learn the intersection of their query with the private set. At the same time, an adversary can simply brute force the private set by systematically querying the full search space. Additionally, as is common for a smaller firm, for cost and simplicity reasons it does not operate its own infrastructure but hosts the service on a third-party cloud provider. However, this provider is not trusted with the confidentiality of the database.

This problem is an instance of Private Set Intersection (PSI). Most cryptographic PSI protocols reason about security within a single-run context and do not consider information leakage across multiple interactions, which leaves brute-force attacks unaddressed.

The canonical defense we discuss in this thesis is rate-limiting the interface, bounding the information leakage that accumulates across multiple client-server interactions. Specifically, we explore the rate-limiting problem in a client-server setting with a disadvantageous threat model summarised in Figure 1.

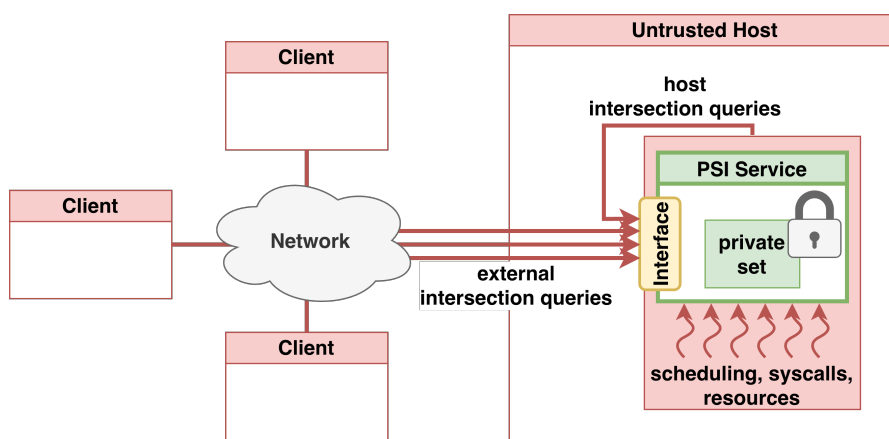


Figure 1: Threat model and problem setting: A private set exposes an intersection interface to clients, including the host. Both clients and the host are untrusted. The host has a privileged position over the PSI service, providing resources, having control over scheduling and memory.

We assume that the [PSI](#) service is hosted by a third party. Potential clients include both external parties as well as parties on the same machine and the host itself. All actors and components outside of the service are *untrusted*. This includes the host OS or hypervisor, which is a *privileged* adversary, since it has full control over the machine on which the service is running.

This makes for a strong adversary, which can read or modify any data the [PSI](#) service is using, allowing it to bypass any rate-limiting logic enforced purely in software by accessing private functions or memory directly. Cryptographic approaches alone are therefore not sufficient to protect against a privileged adversary, and we need a mechanism to isolate the [PSI](#) service from the host. In this thesis we explore how Trusted Execution Environments ([TEEs](#)) as an isolation mechanism can be used to enforce rate-limiting logic against this privileged adversary.

[TEEs](#) are hardware-isolated execution contexts that protect code and data even from a privileged host like the operating system or hypervisor [13, 14]. They can thus provide confidentiality and integrity guarantees for the [PSI](#) service from the untrusted cloud provider. However, they come with some important limitations that make implementing rate-limiting non-trivial.

Typical rate-limiting algorithms accumulate counters or tokens, either system-wide or per-client, and compare them with a given time interval, effectively enforcing a rate of requests per time interval. This requires consistent state (counters) and a trusted time source.

Neither is readily available in a [TEE](#). Popular [TEEs](#) like Intel SGX and AMD SEV-SNP do not provide a time source with the necessary guarantees [4].

Additionally, counters are vulnerable to rollback attacks [11, 26]. The privileged adversary has full control over the scheduling of the [PSI](#) service, allowing it to restart execution from a point where its counter was lower.

Similarly, the host can run multiple clones of the [TEE](#) [11, 17], increasing the effective throughput of its queries.

These are called state-continuity attacks and existing mitigations [16, 17] require additional hardware primitives and increase the [TCB](#) considerably.

CONTRIBUTIONS. First, we analyse why traditional rate-limiting approaches fail in a [TEE](#) environment. We explore mitigations and external primitives that extend the guarantees of the [TEE](#) and their practical limitations.

Second, we discuss Computational Puzzles as a potential alternative that avoids accumulating security-sensitive state and does not rely on any notion of time. Out of several available primitives, we choose Time-Lock Puzzles ([TLPs](#)).

Third, we present a prototype implementation of the TLP-based rate-limiting design in Intel SGX for the PSI example and use it to perform several experiments. Based on the gathered data, we evaluate the security of the implementation and make several real-world examples of possible guarantees based on attacker assumptions.

Finally, we discuss the limitations of this approach, which touch on the fundamental tradeoff between security and usability. We differentiate between scenarios in which our proposed design is more or less suited.

ROADMAP. The remainder of this thesis is structured as follows. Chapter 2 introduces the necessary technical background and introduces the PSI problem formally. Chapter 3 analyses traditional rate-limiting approaches and discusses their limitations in TEEs. Chapter 4 explores computational puzzles as an alternative rate-limiting mechanism while Chapter 5 presents a prototype implementation. Chapter 6 describes the experimental results and evaluates the prototype against the security guarantees of PSI. Finally, Chapter 7 discusses the limitations of the approach, future directions of research and compares the primitives in different settings.

BACKGROUND

The following chapter summarises the background necessary for this thesis. It covers [TEEs](#), rate-limiting and a formal definition of [PSI](#) as our guiding use case.

2.1 TRUSTED EXECUTION ENVIRONMENTS

2.1.1 *Motivating TEEs*

Traditionally, information security is understood as the problem of managing access to data [22]. Data can be *at rest*, *in flight* or *in use* and the use of cryptographic techniques to address the first two have been studied extensively.

However, at some point data must be available in plaintext to compute with it. At that point, the host has full control over the data and can easily read or manipulate it [13, 14]. The increase in cloud computing [7] motivates the need for mechanisms that protect the data *in use*, as the host is not necessarily trusted anymore.

Trusted Execution Environments ([TEEs](#)) were developed to address this issue by establishing isolated execution contexts, typically called *enclaves* or *trust domains*, that protect code and data from the rest of the system stack, including a potentially compromised or malicious host [13, 14]. A key consequence is a reduction in the Trusted Computing Base ([TCB](#)). The [TCB](#) is the minimal set of hardware, firmware, and software that must be trusted and correct for a system’s security guarantees to hold. In a [TEE](#)-based model, the [TCB](#) shrinks to the CPU package and firmware (both signed by the trusted vendor), effectively excluding the OS, hypervisor, and cloud provider from the trust boundary [14, 27].

2.1.2 *Threat Model of TEEs*

Rather than relying on privileged software, [TEEs](#) shift the root of trust to hardware. The core assumption is that all software outside the [TEE](#) is potentially malicious and thus cannot be trusted [13, 27]. In particular, this includes conventionally privileged software like the OS or hypervisor, which are now untrusted.

2.1.3 *Security Guarantees of TEEs*

CONFIDENTIALITY. TEEs guarantee confidentiality of all data they use. While the enclave binary is typically known to the host, the execution state and data processed inside the TEE remain confidential. The host cannot observe which instructions are executing or what values they operate on. Additionally, data leaving the trust boundary is encrypted by dedicated hardware [1, 3, 13, 14].

INTEGRITY. Beyond confidentiality, TEEs also guarantee integrity. Untrusted components cannot tamper with data or the execution of applications in the TEE [1, 3, 13, 14]. In particular, TEE guarantee that they run exactly as intended by the author and that the data protected by the TEE is not modified by the host.

AVAILABILITY. TEEs provide no guarantee for availability. Since the host retains control over scheduling, a malicious host can always interrupt or starve enclave execution [13, 27].

Physical attacks such as probing memory buses or battering ram [13, 18] are out of scope for most TEE designs and thus out of scope for this thesis.

2.1.4 *How TEEs Work*

TEEs vary in their architectural scope. Application-level designs like Intel SGX protect individual processes, VM-level designs like Intel TDX, AMD SEV-SNP and Arm CCA protect entire virtual machines. This section aims to briefly discuss some core mechanisms while giving some concrete examples from popular TEE designs.

Rather than trusting some privileged software to manage access, the CPU itself enforces a distinction between trusted and untrusted execution contexts [13, 14]. The following describes some of the core mechanisms through which this is achieved.

TRUSTED EXECUTION MODES. At the core of any TEE is a dedicated hardware execution mode that the CPU enters when running protected code [13, 14]. Intel SGX, for example, introduces an enclave mode that executes protected code at the user level but isolates it from all other software, including the kernel [14]. Intel TDX takes a different approach by introducing Secure Arbitration Mode, which protects entire virtual machines rather than individual application components [13]. ARM TrustZone partitions the CPU itself into a Secure World and a Normal World, where the processor's current world is reflected in every memory access [4, 14]. In each case, the result is an execution

context whose integrity is enforced by hardware and firmware, not by any components outside of the TCB.

MEMORY PROTECTION. To protect data in use, TEEs reserve dedicated physical memory regions that the hardware prevents any untrusted software from accessing, regardless of privilege level [4, 14]. In application-level TEEs like Intel SGX, this region is called the Enclave Page Cache, and the CPU enforces that only code running in enclave mode can read or write to it [14]. Page table management is left to the untrusted OS, but hardware checks on every memory access verify that each page is mapped only to its authorised virtual address [14].

VM-level TEEs take a different approach. Intel TDX partitions physical memory into normal and secure regions, using per-page metadata to restrict access to software running in Secure Arbitration Mode [13]. AMD SEV-SNP enforces memory isolation through Secure Nested Paging, which tracks the owner of every physical page and ensures that only the assigned VM can write to it [2].

Data leaving the trust boundary is transparently encrypted by dedicated hardware engines, protecting it from components outside the TCB [13, 14]. Intel SGX for example, supports *sealing* of state: When an enclave persists state across sessions, it can seal it to disk using a hardware-derived key bound to the enclave’s identity, ensuring that sealed data can only be recovered by the same enclave on the same platform [14].

ENCLAVE TRANSITIONS. Execution can only enter a TEE through specific hardware-enforced instructions, ensuring that protected code is always invoked at defined entry points [13, 14]. In both application- and VM-level TEEs, the trust boundary is never crossed in an uncontrolled way. Any interruption saves and clears sensitive state before yielding control to the untrusted OS [13, 14].

ATTESTATION. Ensuring integrity of the initial state of an enclave is done through *remote attestation*, where the CPU generates a signed measurement of the enclave’s contents, typically a hash of its code and initial data [13, 14]. A remote party can verify this measurement against a known, good value and confirm that the enclave has not been tampered with before provisioning it with secrets [14, 26]. This mechanism ensures that a client can trust that the expected code is actually being run by the host.

2.1.5 Limitations of TEEs

Despite their strong security guarantees, TEEs have several important limitations that follow directly from their design and threat model assumptions.

NO DENIAL OF SERVICE PROTECTION. As the threat model does not guarantee availability, Denial of Service attacks fall outside the protection scope of most TEE designs [11, 13, 27]. This means that any security policy relying on the continuous or timely execution of enclave code cannot be enforced by the TEE alone.

TRUSTED TIME. TEEs generally lack a trusted time source [4]. In practice, enclaves must either request a Time Stamp Counter (TSC) from the host or rely on platform services, both of which are outside the TCB [4, 6, 27]. A malicious OS can therefore manipulate the time perceived by an enclave, either by skewing system clocks or by selectively scheduling enclave threads to induce artificial delays [4, 6]. Alder et al. formalise time guarantees into five levels [4].

At **T₀**, no guarantee is given at all, while **T₁** guarantees that time monotonically advances.

T₂ guarantees that time moves at a constant, known pace, providing a lower bound on elapsed wall-clock time even if the received timestamp is not guaranteed to be fresh.

T₃ additionally guarantees that the timestamp is read with a known delay, and **T₄** that the use of time is atomic.

Table 1 shows where several common TEE designs fall within this hierarchy [4].

TEE	Level	Mechanism
Intel SGX	T ₀	unprotected TSC
Intel TDX	T ₃	TSC virtualisation
Arm TrustZone	T ₄	hardware-assigned secure time device
Arm CCA	T ₃	direct access to TSC registers
AMD SEV-SNP	T ₀	Secure TSC not guaranteed to be tamper-resistant

Table 1: TEEs and their available time source

Remark. Intel SGX with *Platform Services* does provide a **T₃** time source, but it has been discontinued.

Alternative sources for a trusted remote time source include a trusted third party time server or TPMs.

A TPM is a dedicated hardware security co-processor that provides a hardware-rooted foundation for system integrity and attestation [14, 23]. They expose two time-related values: *Time*, a millisecond counter since last power-on, and *Clock*, a monotonically increasing value derived from *Time* that cannot be rolled back within a single power cycle [25]. The *Time* value in particular can serve as a **T₄** time source from the TPM perspective.

STATE CONTINUITY. TEEs protect the integrity of data in use, but they struggle to guarantee the freshness of state [11, 17, 26]. As mentioned before, when an enclave seals data to disk, it uses a hardware-derived key to encrypt and authenticate it. However, this sealing mechanism does not provide freshness guarantees [14, 17, 26]. A malicious OS can snapshot the sealed state at any point and later replay it, replacing the current version with a stale but authentic older version. This is known as a *rollback attack* [11, 26].

The standard mitigation is to use Monotonic Counters (MCs). MCs are tamper-resistant hardware components that maintain a value which can only ever be incremented, never reverted to a previous state [16]. By storing the current MC while sealing its state and incrementing the MC, the enclave can later detect stale state, because it will have an outdated MC value stored.

Hardware monotonic counters are available via TPMs, which also expose increment-only NVRAM backed counters. Increment latency is in the order of 17-97ms and write endurance is limited [16]. ADAM-CS improves on this via a two-tier architecture of fast in-memory virtual counters backed by physical TPMs [16]. ROTE takes a different approach by distributing counter state across enclaves on multiple hosts [17].

NO INSTANCE UNIQUENESS. Since TEEs cannot enforce or track how many instances of a given enclave binary are launched by the host, an adversary can run multiple copies of the same enclave simultaneously and partition inputs across them [11, 17]. Because each instance produces a valid attestation, a remote party has no way to detect that multiple instances are running [11, 26]. This attack is called a *cloning attack*. In addition, since the keys used to authenticate sealed state usually depend on the enclave measurement, it will be accepted by any clone.

Proposed mitigations include *CloneBuster*, a detection mechanism that operates without a trusted third party [11]. It exploits a cache-based covert channel: enclaves with the same binaries signal their presence to one another by accessing shared cache lines, and detect clones by observing the resulting latency increase from competing accesses. This achieves an F1 score of 0.999, correctly flagging almost all clones [11].

SIDE-CHANNEL VULNERABILITIES. TEEs do not protect against micro-architectural side-channel attacks, as enclaves share hardware resources such as caches and branch predictors with untrusted code on the same machine [13, 14]. An attacker can therefore infer information about enclave execution by observing timing differences or cache access patterns from outside the trust boundary.

2.2 RATE LIMITING

Rate limiting is a control mechanism that restricts how frequently a client can perform a given operation within a specified time window [15]. It is commonly used to defend against brute-force attacks, where an attacker repeats a security-sensitive operation arbitrarily fast to enumerate possible inputs [4, 15]. Classical algorithms such as token bucket, fixed window counter, and sliding window counter differ in their trade-offs, but all track state over time: they maintain a counter of accepted requests and compare it against elapsed time to enforce a maximum rate [15].

2.3 PRIVATE SET INTERSECTION

The core motivating application for this thesis is the Private Set Intersection (PSI) problem. In this setting, a trusted party holds a private set

$$\mathcal{S} = \{s_1, \dots, s_n\} \subseteq \mathcal{U}$$

drawn from a universe $\mathcal{U}$ and exposes an interface that allows untrusted parties to check which elements of a submitted set are contained in $\mathcal{S}$:

$$\text{QUERY}(E = \{e_1, \dots, e_l\}) = E \cap \mathcal{S} = \{e_i \in E : e_i \in \mathcal{S}\}$$

The untrusted parties should learn only the result of their query and nothing about the remaining elements of the private set.

Note that this is different from the well-studied cryptographic protocols often associated with PSI.

Formally, we define three guarantees necessary for the security of a [PSI](#):

Label	Guarantee
P1:	$\mathcal{S}$ is confidential, there is no other way to learn its content than to use the designated interface.
P2:	The information learned through the interface is exactly the intersection of the requested elements with the private set.
P3:	Full enumeration of the universe $\mathcal{U}$ through the interface is <i>sufficiently costly</i> .

Table 2: Security Guarantees of Private Set Intersection

Remark. We say that full enumeration is *sufficiently costly*, if it exceeds the cost a realistic attacker is willing to expend given the expected benefit.

This is intentionally left vague, as it depends heavily on the exact use-case and threat model.

We are considering the problem of [PSI](#). Specifically, we consider a scenario where the service containing the private set $\mathcal{S}$ and the interface allowing intersection queries, runs on an untrusted host. This means that we consider a potentially privileged adversary. The properties of [PSI](#) we want to satisfy in this environment are summarised in [Table 2](#).

The focus of this thesis is on [P3](#). A malicious client can exploit the query interface to enumerate the entire universe, submitting every possible value until the private set is effectively reconstructed [\[26\]](#). This threat is particularly severe when inputs are low-entropy or predictable, such as 4-digit PINs or phone numbers, since the search space is small enough to be exhaustively queried in a reasonable amount of time.

Rate limiting is therefore a necessary component to fulfill [P3](#). By bounding how many queries a client can submit, the cost of full enumeration becomes prohibitively high, even if each individual query is answered correctly [\[26\]](#).

This chapter will explore different approaches and explain their respective shortcomings.

3.1 ATTEMPT 1: PURE CRYPTOGRAPHY

Since the host has access to the $\mathcal{S}$ stored in its memory, we cannot stop it from accessing it.

To address [P1](#), we can store the hashes of the elements in $\mathcal{S}$. Using a cryptographic hash function guarantees that the host cannot reverse engineer the original elements that resulted in the stored hashes. The client-facing interface would then compute the hashes of the requested elements and compare those to the private set, similarly to how passwords are canonically stored and compared in hash form.

3.1.1 Limitation

Firstly, storing hashes of the elements still leaks the information of the size of $\mathcal{S}$ to the host.

Secondly, the host can access the private set in memory directly, bypassing the defined interface and violating [P1](#).

Thirdly, the host can thus also easily bypass any rate-limiting logic placed on the interface, making it impossible to enforce any constraints regarding [P3](#).

3.2 ATTEMPT 2: RATE-LIMITING

Clearly, we need at least a guarantee that the interface and any included rate-limiting cannot be bypassed by the host. As described in Section 2.1.3 TEEs provide exactly this guarantee.

From now on, all implementations are assumed to be running in a TEE.

Working in the threat model of TEEs, **P1** and **P2** can be addressed by storing $\mathcal{S}$ in the protected memory region and correctly implementing an interface that avoids leaking any information through side-channels.

Clients as well as the host now have to go through the interface, where we can also implement rate-limiting.

A natural approach would be to use a rate-limiting algorithm like Algorithm 1.

Algorithm 1 Rate-Limiting PSI

```

1: counter  $\leftarrow$  0
2:  $t_1 \leftarrow \text{CURRENT\_TIME}()$ 
3: function REQUEST( $E$ )
4:    $t_2 \leftarrow \text{CURRENT\_TIME}()$ 
5:   time_interval  $\leftarrow (t_2 - t_1)$ 
6:   if time_interval  $\geq$  EPOCH then            $\triangleright$  New epoch has started
7:      $t_1 \leftarrow t_2$ 
8:     counter  $\leftarrow$  0                        $\triangleright$  Reset counter
9:   end if
10:  if counter +  $|E| \leq$  MAX_COUNT then        $\triangleright$  Under the rate limit
11:    counter  $\leftarrow$  counter +  $|E|$ 
12:    return  $E \cap \mathcal{S}$ 
13:  else
14:    return Reject
15:  end if
16: end function

```

The algorithm counts the number of requested elements it accepted in the last EPOCH and compares that to the allowed number MAX_COUNT. For that reason, counter is increased by the number of elements in the input, whenever a request is accepted. This guarantees that the number of requested elements per EPOCH is at most MAX_COUNT, i. e. the following rate:

$$r = \frac{\text{MAX_COUNT}}{\text{EPOCH}}$$

For the purpose of this thesis, this algorithm is equivalent to more sophisticated rate-limiting algorithms and illustrates the role of counters and time more clearly.

3.2.1 Security Analysis

Using Algorithm 1, we can fulfill **P3** in the following way. First we observe that full enumeration of $\mathcal{U}$ will take $t = \frac{|\mathcal{U}|}{r}$ epochs. We can now define a time duration τ that, if it were the minimal required time for full enumeration, we consider to be sufficiently costly. By setting $r \leq \frac{|\mathcal{U}|}{\tau}$, **P3** is satisfied.

This requires `current_time()` to provide a **T2** time source, guaranteeing that any measured time interval from `current_time()` is *at least* as long as the wall-clock time interval:

$$\begin{aligned} (t_2 - t_1) &\geq \text{EPOCH}_{\text{wall_clock}} \\ \implies \\ r &= \frac{\text{MAX_COUNT}}{\text{EPOCH}} = \frac{\text{MAX_COUNT}}{(t_2 - t_1)} \leq \frac{\text{MAX_COUNT}}{\text{EPOCH}_{\text{wall_clock}}} \end{aligned}$$

3.2.2 Limitation

As summarised in Table 1, while many **TEEs** provide native **T2** time sources, some popular ones like Intel SGX and AMD SEV-SNP do not.

Without it, the host can simply speed up the time for the enclave by providing timestamp values that are actually in the future. Speeding time up by a factor N achieves a new request rate of $N \cdot r$:

$$r_{\text{time_attack}} = \frac{\text{MAX_COUNT}}{\text{EPOCH}_{\text{time_attack}}} = \frac{\text{MAX_COUNT}}{\frac{1}{N} \cdot \text{EPOCH}} = N \cdot r$$

This clearly violates **P3**, since we cannot set a lower bound τ on full enumeration.

3.3 ATTEMPT 3: RATE-LIMITING + TPM

For **TEEs** lacking a native **T2** time source, a **TPM** can serve as a substitute. The **TPM's** *Time* value satisfies trusted remote time and results in **T2** level from the point of view of the **TEE**. The **TPM** can be queried via `TPM2_ReadClock` for local use [25]. Figure 2 shows an exemplary simple protocol to establish a secure channel through the untrusted host.

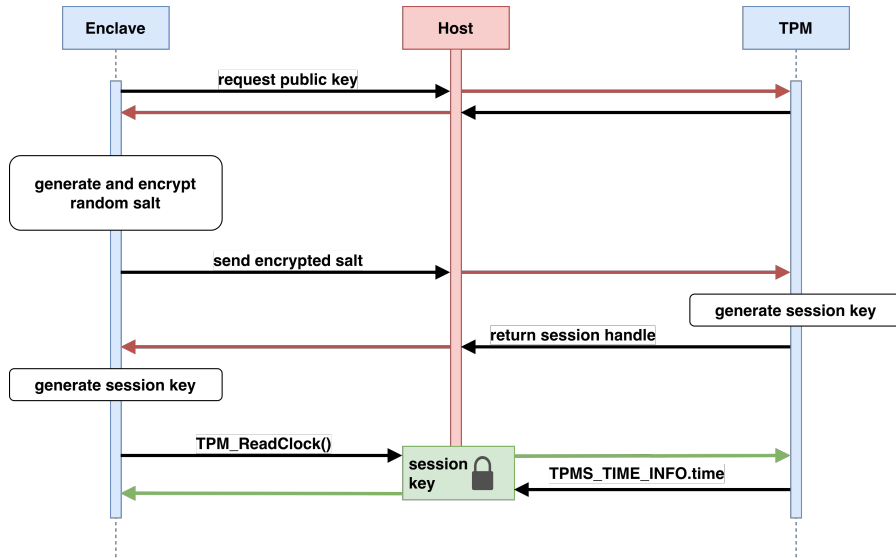


Figure 2: Protocol to establish a secure channel between enclave and TPM for trusted time retrieval. Note that the TPM’s public key is hardware backed and certified by the manufacturer, allowing the enclave to detect a spoofed or substituted TPM.

Adding the channel setup to the initialisation phase of the TEE and plugging in the TPM query in place of `current_time()` in Algorithm 1, removes the need for a native T₂ time source.

With a T₂ time source available via the TPM, Algorithm 1 satisfies P₃, provided the counter remains consistent.

3.3.1 Limitation

Querying the TPM is slow: access latencies are around 32 ms on average [16] making fine-grained rate-limiting difficult. Furthermore, the TPM is a shared resource with limited throughput, meaning frequent queries from multiple applications can degrade performance further [4].

Additionally, a clock drift of up to $\pm 32.5\%$ is allowed as per the TPM 2.0 specification [25], but this can easily be corrected for when defining the allowed rate.

Apart from the limitations of TPMs as a time source, the issue of state continuity is still unaddressed: The counter variable is *security-sensitive state*, as any inconsistency in it bypasses the security guarantees. As mentioned in Section 2.1.5, such state is vulnerable to rollback attacks.

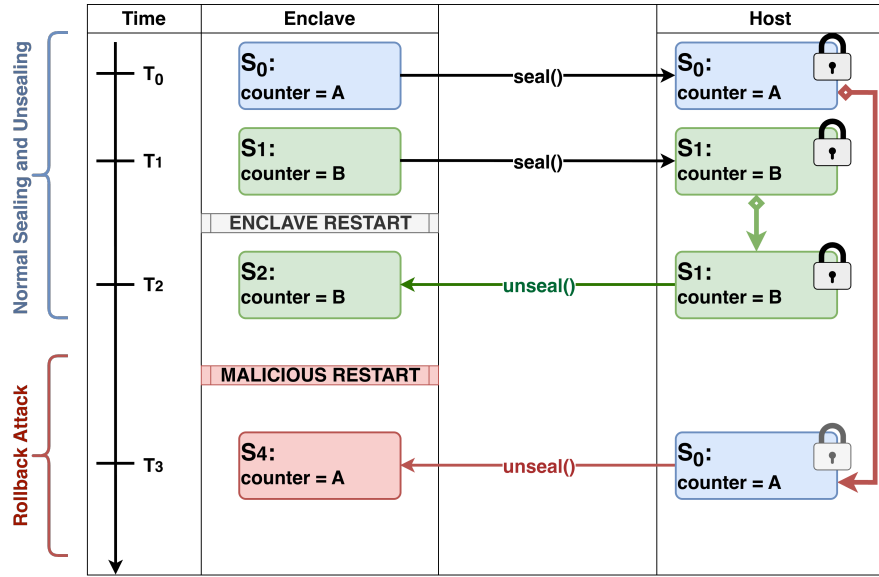


Figure 3: Rollback attack: setting the enclave back to previous state by replaying stale state

Figure 3 visualises such an attack. Essentially, the enclave gets set back to a stale, outdated state.

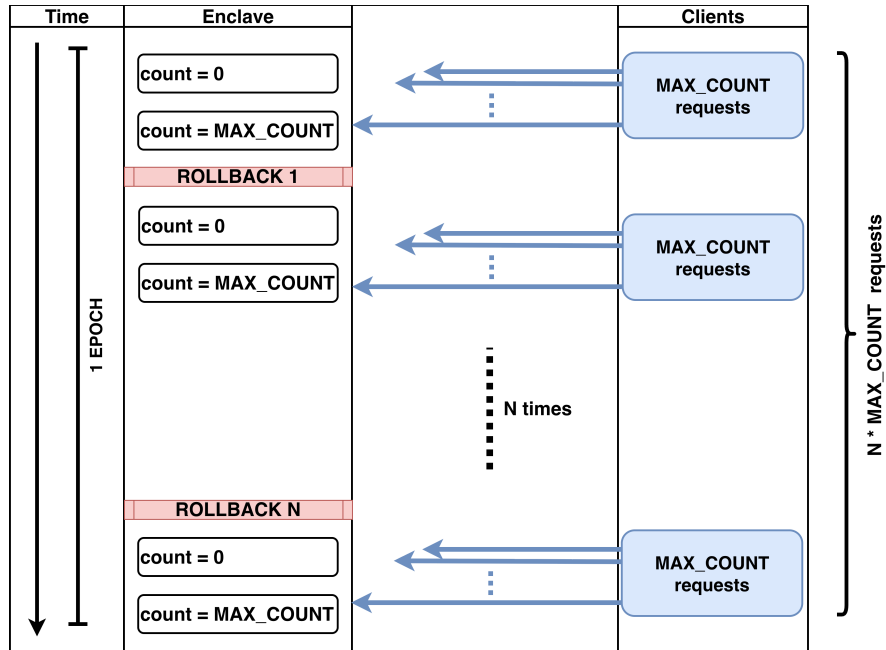


Figure 4: Rollback Example: rolling back to reset the counter to zero

Figure 4 shows a possible way of using this attack to bypass the security guarantees of the rate-limiting algorithm 1.

For a rollback attack as illustrated in Figure 4, the attacker can obtain a new effective rate of

$$r_{\text{rollback}} = N \cdot r$$

assuming it can rollback the enclave state N times in one EPOCH.

More precisely, assuming a rollback takes R time to execute, the attacker can roll back

$$N = \left\lfloor \frac{\text{EPOCH}}{R} \right\rfloor$$

times in one EPOCH. This factor N violates the lower bound τ for full enumeration, thus [P3](#) is no longer satisfied.

While scaling down both EPOCH and MAX_COUNT by a factor M reduces the advantage

$$r_{M_rollback} = \left\lfloor \frac{\frac{1}{M} \cdot \text{EPOCH}}{R} \right\rfloor \cdot \frac{\text{MAX_COUNT}}{\text{EPOCH}} = \frac{1}{M} \cdot r_{rollback}$$

there is a practical limit to how much we can reduce EPOCH and MAX_COUNT. In the extreme, MAX_COUNT becomes small enough to refuse many legitimate requests with large input sets. Additionally, R is determined by the attacker's rollback implementation and is outside our control, so the remaining advantage $r_{M_rollback} = \frac{1}{M} \cdot r_{rollback}$ has no guaranteed lower bound.

3.4 ATTEMPT 4: RATE-LIMITING + TPM + MC

Monotonic Counters (MCs), as introduced in Section 2.1.5, can detect rollback attacks by storing the current MC value alongside the counter. On update to counter, the MC is incremented and on read, the TEE verifies that the stored MC value matches the expected value, rejecting any inconsistencies.

With a MC protecting the security-sensitive state, Algorithm 1 is no longer vulnerable to rollback attacks. In particular, the counter cannot be reduced by an attacker and thus the effective rate r of the algorithm cannot be increased.

3.4.1 Limitation

Securing the counter with an MC requires a reliable MC primitive. However, all practical constructions introduce significant limitations.

TPMs provide hardware-rooted counters, but the high increment latency of limited NVRAM write endurance make them unsuitable for high-frequency rate-limiting [16, 17].

ADAM-CS addresses this through a two-tier architecture of in-memory virtual counters backed by physical TPMs [16], but grows the TCB substantially. ROTE distributes counter state across enclaves on multiple hosts [17], but depends heavily on network latency and scale.

Furthermore, MCs do not protect against cloning attacks. As [11] explains, the increment and read of an MC cannot be guaranteed to be atomic, so a cloning attacker can bypass any verification by scheduling

all the increments of the clones before the first one reads its value again.

Figure 5 shows the high level idea behind a cloning attack against our rate-limiter.

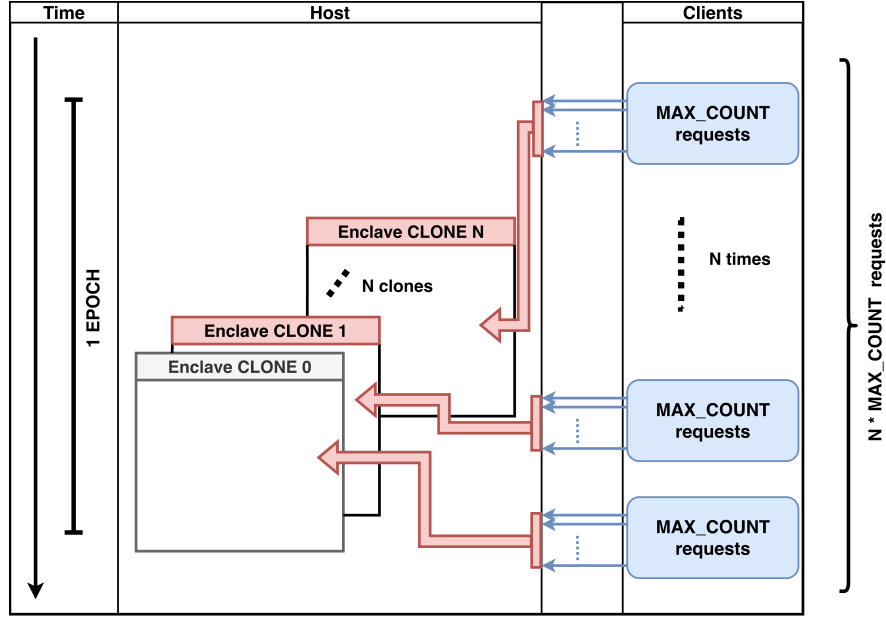


Figure 5: Cloning Attack: each clone gets handed a fraction of the requests

An attacker creating N clones of the rate-limiting enclave can redirect client requests, essentially reducing the requests arriving at any single enclave by a factor of N . Thus the rate is $r_{\text{cloning}} = N \cdot r$

And as with other attacks, this factor N results in **P3** not being satisfied.

Remark. To address cloning attacks, we can add CloneBuster [11] as a detection mechanism to the TEE running Algorithm 1. If a clone is detected, the enclave stops allowing new requests.

However, this increases the TCB further. Additionally, in a multi-socket environment like Intel SGXv2, clones on different sockets don't share a cache [24], making detection across different sockets infeasible.

3.5 SUMMARY OF STRAWMAN SOLUTIONS

Since our threat model assumes a privileged adversary, we need the guarantees provided by TEEs to enforce a restricted interface. This allows us to introduce rate-limiting to satisfy **P3**. However, rate-limiting in TEE is non-trivial. Each attempt introduced an additional external primitive to patch a gap left by the previous one. A TPM provides a T2 time source, MCs protect the counter against rollback attacks, and CloneBuster attempts to detect cloning. Table 3 summarises these attempts and the attacks they address.

Attempt	Addressed Attacks				
	Bypassing Interface	Time Manipulation	Rollback	Cloning	
1: Pure Cryptography	×	×	×	×	
2: Rate-Limiting (TEE)	✓	×	×	×	
3: Rate-Limiting + TPM	✓	✓	×	×	
4: Rate-Limiting + TPM + MC	✓	✓	✓	(✓)	

Table 3: Summary of strawman attempts and the attacks they address.

A comprehensive solution requires all three simultaneously, growing the [TCB](#) considerably and introducing significant latency, operational overhead, and complexity. [TPMs](#) do not scale well and other primitives like *ADAM-CS*, *ROTE* and *CloneBuster* should be considered research projects, none of them are drop-in libraries and their complexity has hindered widespread adoption.

The following chapter explores an alternative approach that avoids these primitives entirely.

RATE-LIMITING ALTERNATIVES

Chapter 3 showed that we need TEE guarantees to protect our PSI service from the host. In order to satisfy P_3 , we also need some form of rate-limiting on the interface, but counter- and EPOCH-based algorithms are difficult to implement securely in a TEE. In particular, our design cannot practically rely on a *secure time* source and it cannot hold any *security-sensitive state*.

This chapter aims to explore alternative methods of achieving rate-limits under these constraints.

4.1 COMPUTATIONAL PUZZLES

For this, we need to broaden our understanding of rate-limiting. Previously, we looked at rate-limiting as enforcing a time delay to avoid clients making more requests than wanted. We used these delays to guarantee that full enumeration of the private set takes a certain amount of time, making it sufficiently costly and thus fulfilling P_3 .

Another way of restricting the number of requests a client can make is by making them expend computing resources. This area of research is sometimes called *Computational Puzzles* and broadly speaking, we can group them into three categories: parallel Proof of Work (PoW), Memory Hard Functions (MHFs) and Sequential Functions.

4.1.1 Puzzle-Based Rate-Limiting Design

The high level design of this category of rate-limiting can be seen in Figure 6.

The client is expected to solve a puzzle in order to be allowed to make a request. The proof that the puzzle has been solved is appended to the request, allowing the rate-limiter to verify it. Only after that succeeds is the request forwarded to the actual service. This essentially shifts the rate-limiting mechanism outside the enclave, since the computation is done by the client and the enclave only verifies. The computing cost associated with solving the puzzle is what we will use to satisfy P_3 .

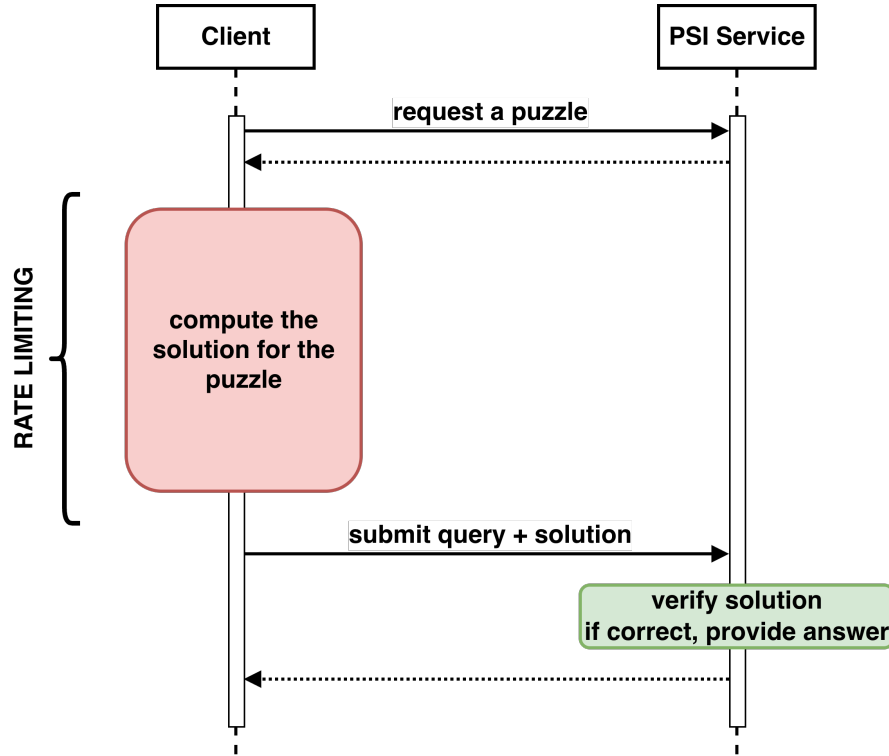


Figure 6: High-level design of rate-limiting using computational puzzles

IN-TEE ENFORCEMENT. A simple alternative to client-side puzzles is a busy loop inside the TEE. The enclave spins for a fixed number of iterations per element before returning the result, which cannot be bypassed and requires no client cooperation. However, the computational cost falls entirely on the TEE operator rather than the client. In settings where the host bears the computational cost and is simultaneously the sole client, this is a valid approach. It can make equivalent guarantees as the primitives discussed below and is equally secure against the attacks mentioned in the previous chapter.

However, this thesis assumes a more general deployment setting and thus focuses on client-side approaches.

4.1.2 Work-Depth Model

In order to quantify this cost, we borrow the ideas of the *Work-Depth model* formalised by Blelloch in [8] and normally used to analyse the efficiency of parallel algorithms. The model expresses computational cost as a pair $(\mathbf{W}, \mathbf{D})$, where $\mathbf{W}$ captures the total amount of work and $\mathbf{D}$ the depth, i. e. the sequential part that cannot be parallelised. Table 4 shows the precise definitions we will use to argue **P3**.

Dimension	Definition
W	The number of CPU cycles expended to solve a puzzle.
D	The number of <i>sequential steps</i> necessary to solve a puzzle.

Table 4: Definition of the domains of the work-depth model.

Remark. Mapping *sequential steps* to a precise unit like number of CPU cycles is non-trivial and will be explored in Chapter 6.

EXAMPLE. If we assume $W_{\min}$ total cycles needed to fully enumerate $\mathcal{U}$ is considered sufficiently costly, then a puzzle with computational cost $W \geq \frac{W_{\min}}{|\mathcal{U}|}$ satisfies **P3**. Similarly, we can define $W_{\min}$ as a sufficiently costly number of sequential steps, and design a puzzle with $D \geq \frac{D_{\min}}{|\mathcal{U}|}$.

Remark. Which dimension is chosen depends heavily on the capabilities of the attacker model and our definition of infeasibility. Concrete examples are explored in Section 6.3.2.

4.2 PROOF OF WORK

Proof of Work (**PoW**) puzzles are probabilistic by nature. Solvers repeatedly hash candidates until one satisfies a target condition, with no smarter strategy than brute force. A canonical example is Bitcoin [19], where the solver searches for a nonce such that the hash falls below a target threshold. Crucially, **PoW** schemes are inherently parallel. A solver can try many candidates simultaneously, so more hardware directly translates to more attempts per second.

4.2.1 Quantitative Guarantee

PoWs allow for a guarantee of a lower bound on **W**. As mentioned in the beginning of this chapter, this can be used to satisfy **P3**, by making **W** large enough so that the cost of the required hardware, electricity and infrastructure makes full enumeration sufficiently costly, even when full parallelisation were possible.

However, since no sequential guarantee is given, no bound can be achieved on **D**.

4.3 MEMORY HARD FUNCTIONS

Memory Hard Functions (**MHF**s) are hash functions whose evaluation requires a significant amount of memory. It is typically used in the

context of storing password hashes to make brute-force attacks more expensive, since specialised parallel hardware usually cannot handle large memory demands well [20].

Attempt 1 in Section 3.1 stored the private set in hash form. Making this a memory hard hash function would force a brute-force attacker to consume considerably more memory per evaluation.

4.3.1 Quantitative Guarantee

MHFs offer a guarantee on a dimension not captured by the (W, D) model, namely on the amount of memory. However, MHFs are designed to address a specific adversary which uses specialised hardware like GPUs, where memory is a real constraint because such hardware cannot easily scale memory alongside compute. In our threat model clients and hosts do not have such specific memory constraints and thus the guarantees that MHFs provide are not relevant for this thesis.

4.4 SEQUENTIAL FUNCTIONS

Sequential Functions are, as the name suggests, functions that require a certain number of sequential computational steps to evaluate. Unlike PoWs, they are specifically designed to eliminate any parallel advantage. This guarantees that computing the function cannot be sped up beyond a certain lower bound, no matter the parallel resources.

4.4.1 Verifiable Delay Functions

The most well known formalisation of this idea are Verifiable Delay Functions (VDFs), introduced by Boneh et al. [9]. A VDF guarantees that evaluating $f(x)$ requires at least T sequential steps even with unbounded parallelism, while also producing a short publicly verifiable proof that allows anyone to check the result in significantly less time than the T steps [9]. Efficient constructions were later proposed by Wesolowski and Pietrzak using groups of unknown order [10].

4.4.2 Time-Lock Puzzles

A closely related and simpler concept are TLPs, introduced by Rivest et al. [21]. The canonical construction for TLPs is repeated squaring modulo N , where $N = p \cdot q$ and p, q are large primes:

Given x , compute

$$y = x^{2^T} \bmod N$$

where N is a RSA modulus.

The creator of the puzzle, can use Euler’s theorem together with knowledge of the factorisation of N to verify the result efficiently, without redoing the computation:

$$x^{2^T} \bmod N = x^e \bmod N$$

where $e = 2^T \bmod \phi(N)$ and $\phi(N) = (p-1)(q-1)$.

The security of this puzzle relies on the assumption that factorising N into its prime factors p and q is computationally infeasible. Without knowing them, there is no efficient known way of computing $\phi(N)$, which results in the need to perform the T squarings sequentially [21].

TLPs are a strictly weaker primitive than VDFs, since they lack the short publicly verifiable proof. As Boneh et al. note, this is precisely the property that differentiates them [9].

4.4.3 Quantitative Guarantee

Sequential Functions like VDF and TLPs can guarantee some number of required total cycles $W_{\min}$ to solve the puzzle, thus satisfying P3 in the same way as PoW by enforcing a lower bound on W .

However, the sequential nature of them provides an additional guarantee on the depth D of the computation. In particular, the T squarings have to be performed sequentially, regardless of available hardware, which means we can also enforce a lower bound on D .

This is a strictly more powerful guarantee than that of PoWs, which only address the W dimension.

4.5 TIME LOCK PUZZLES FOR PSI

Since Sequential Functions can make guarantees on both the W and D dimension, they are more versatile and thus the most suited of the computational puzzles. Additionally, we do not need public verifiability. It is enough to be able to quickly verify solutions internally.

Unlike delay-based rate-limiting, which enforces a rate on the interface regardless of which elements are requested, a TLP-based design enforces a cost per element learned. This is a better fit for the PSI setting, where the threat is enumeration of specific elements, not frequency in which the interface is called.

For that reason, the chosen primitive of this thesis are TLPs. In the following, some design considerations and consequences, as well as the security guarantees are discussed.

4.5.1 Stateless Design

A practical design cannot rely on security-sensitive state in order to be secure against rollback attacks. This means we cannot store any

per-request or per-client state in the design. TLPs allow us to achieve this by deriving the base x deterministically from the input, each time a request is made or a solution is submitted. In particular, it is not dependent on the client or a nonce, but rather deterministically computed based on the requested elements alone. This has several important consequences.

REPLAY OF SOLUTION. The derivation being deterministic means that a solution y can be replayed and will always be accepted for the same requested elements. Reusing the same solution only works for the same input set, so no new information is learned.

SHARING OF SOLUTION. Another important observation is that x is independent of the client. A solution for an input can thus be shared with other clients, which can reuse it to query the private set without performing any squarings. However, there are two reasons, why this is not relevant in our setting.

Firstly, if clients collaborate, they can also share the information $E \cap S$ directly. No further benefit comes from sharing a solution, since as discussed, it only works for that specific input.

Secondly, going back to the threat model defined in Section 2.1.2, the host has the capabilities to spoof any identifiers associated with a request anyways, since it is in full control of the communication paths.

ROBUSTNESS. Not having to store any per-client or per-request state makes the design much simpler and more scalable. This is especially relevant for a TEE setting, where memory space is usually fairly limited [14]. An added benefit of this choice is more robustness against Denial of Service attacks that specifically aim at overflowing the "per-open-request-state" memory.

SECURITY CONSEQUENCE. The stateless design addresses a key attack identified in Chapter 3. Without any accumulating security-sensitive state, rollback attacks have no effect. There is no counter that can be reverted. The intersection is only provided, if a valid solution tied to the specific elements has been received. No rolling back to any execution state allows for learning new information without first having provided the respective solution to the puzzle.

DEPTH GUARANTEE. Since we do not have persistent state, we cannot enforce any limits across multiple requests. This allows for the following attack: An attacker can request N elements and compute N puzzles in parallel. Generation and verification of the puzzle is fast, so these request can be processed by the rate-limiter at a much quicker rate, thus resulting in a total enumeration cost of $(|\mathcal{U}| \cdot \mathbf{W}_{\min}, \frac{1}{N} \cdot |\mathcal{U}| \cdot \mathbf{D}_{\min})$.

In other words, different to **W**, the **D** guarantee does not extrapolate from single requests to many.

This highlights that the lower bound on **D** is a per-element bound, not a system-level one and the depth guarantee should be understood as a lower bound on the cost *per element* rather than a guarantee on the cost of full enumeration.

4.5.2 Two-Phase Protocol Design.

Another core design consequence are the two distinct phases of the protocol. In a first phase, the puzzle must be requested and/or communicated to the client and in the second phase, they can submit the final request with the proof attached. This means that a client must "play along" and adapt this protocol whenever such a rate-limiter is involved. The client has to fundamentally agree to perform the extra steps associated with solving the puzzle. And while it would be possible to fix N , T and the derivation of x and to make it public knowledge to the clients, there are some advantages to redefining it per request.

Firstly, it means that they can be changed later. For example, it would be easy to change the modulus if $\phi(N)$ or the exponent were to get compromised.

Secondly, and more importantly, as will be explored in Chapter 6, this flexible approach allows us to fine-tune the security parameter T per request and dependent on some dynamic information.

4.5.3 Security Guarantees

TIME MANIPULATION. No step of the TLP-based rate-limiting requires any notion of time, thus making a **T2** time source unnecessary.

ROLLBACK. As mentioned above, a TLP-based rate-limiter does not store any per-request or per-client state, thus making rollbacks useless.

CLONING. Using computational puzzles offloads the rate-limiting to the client. Thus cloning the enclave has no effect on the amount of work **W** that has to be performed by the client for a request. Implementation-specific effects of cloning on the **D** dimension are discussed in Section 6.2.2.

4.6 SUMMARY OF ALTERNATIVE RATE-LIMITING

As discussed in this chapter, Computational Puzzles offer an alternative approach to rate-limiting that avoid the primitives required by traditional algorithms. Instead of relying on a consistent counter and a

trusted time source, the cost of making requests is shifted to the client, who must expend computing resources.

Primitive	Guarantees	
	W	D
Proof of Work	✓	×
Memory Hard Functions	×	×
Sequential Functions	✓	(✓) per-element

Table 5: Summary of computational puzzles and their guarantees.

Table 5 summarises the different primitives explored and their respective guarantees.

The inherent parallelism of PoWs makes them less versatile for our PSI use case.

Sequential Functions address this by enforcing a sequential lower bound on the computation. VDFs provide sequentiality, efficiency, and public verifiability. However, public verifiability is not a requirement for our use case.

TLPs are a simpler primitive that provides exactly what we need: a guarantee that solving the puzzle requires T sequential squarings, with no parallel shortcut available to the solver.

TLPs are therefore the most suitable primitive for our needs. The following chapter explores a concrete implementation of a TLP based rate-limiter, designed for Intel SGX.

IMPLEMENTATION OF TIME-LOCK PUZZLES

Chapter 4 introduced Time-Lock Puzzles (TLPs) as an alternative to time-based rate-limiting schemes that does not rely on any additional primitives. This chapter discusses the design choices of a prototype implementation of rate-limiting a Private Set Intersection (PSI) using TLP. In addition, it describes the experimental setup, with the results examined in Chapter 6.

5.1 IMPLEMENTATION

The prototype was implemented in C++ for Intel SGX using the Intel SGX-SDK framework. The rate-limiting logic adds roughly 200 lines of code, consisting of crypto initialisation, puzzle generation, parameter configuration and puzzle verification.

For the cryptographic operations and "big number" support, the *mbedTLS* library is used. It provides the necessary primitives like the generation of large primes, arithmetic operations on large numbers and modular exponentiation.

5.1.1 Environment

Component	Version/Details
Hardware	Intel Core i9-9900K CPU @ 3.60GHz
Architecture	x86_64
Intel SGX-SDK	Version 2.23.100.2
<i>mbedTLS</i>	Version 3.5.0 (Intel SGX build)

Table 6: Implementation Environment

5.1.2 Architecture

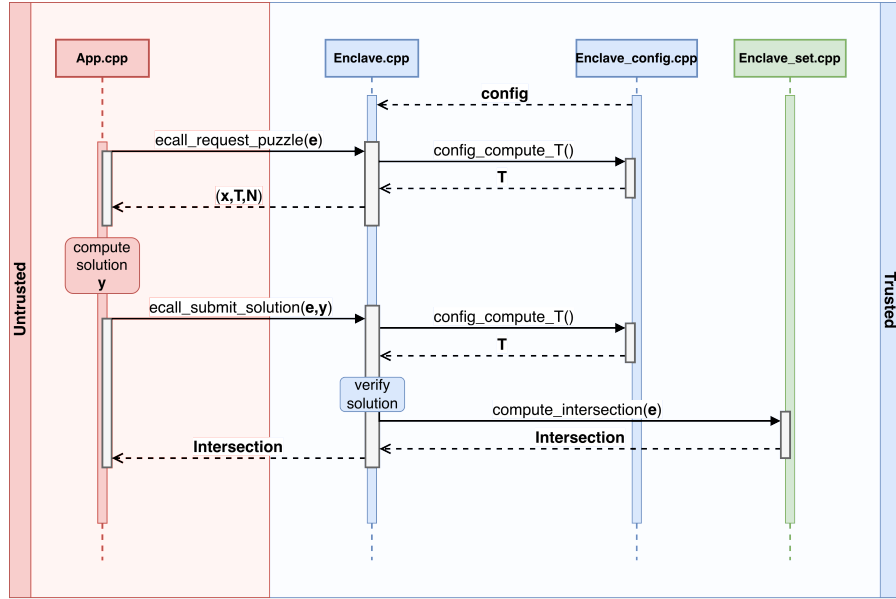


Figure 7: Design of the Intel SGX TLP implementation.

Figure 7 shows the high level design of our implementation. The trusted Enclave exposes an interface consisting of `ecall_request_puzzle` and `ecall_submit_solution`. The rate-limiting logic is enforced inside the Enclave and is thus protected. It holds the cryptographic state, generates the puzzles and verifies solutions. If a solution is accepted, the request gets forwarded to the private set implementation, which is also running inside the Enclave.

To test whether an element is in the private set, the App first asks the Enclave for the relevant TLP details. After the App has received the puzzle, it can solve it asynchronously and submit another request, this time including the solution to the puzzle.

5.1.3 Design Choices

DETERMINISTIC PUZZLE GENERATION. The base x for the puzzle is derived deterministically using the `mbedtls_sha256` hash function:

$$x = \text{sha256}([e_1, e_2, \dots, e_n])$$

The requested elements are sorted first, mirroring the set semantics of the input E .

TWO-PHASE PROTOCOL. The two-phase protocol is implemented by exposing two different functions. `ecall_request_puzzle` is the first phase, which returns a puzzle to the client. As the next section will

explore, we can provide different puzzle parameters to tweak the difficulty.

`ecall_submit_solution` allows the client to make a query to the private set. It appends the solution to the puzzle and receives the response.

PRECOMPUTATION OF THE EXPONENT. `phi` and the exponent are precomputed at initialisation time. They never leave the Enclave memory, so they stay secret from the host. The exponent is used in the verification to quickly compare solutions.

5.1.4 Configurations

The implementation allows for some exemplary configurations. These are by no means exhaustive but should be understood as some possible directions. While the prototype allows changing the configuration at runtime using `ecall_set_config`, a production implementation would fix it at compile time.

Configuration	Explanation
Varying T	configure difficulty by adjusting the number of squarings per element
Linear Scaling	a request consisting of n elements requires $n \cdot T$ squarings
Expected Cycles	T is set such that randomly finding an element in $\mathcal{S}$ by probing $\mathcal{U}$ costs an expected minimum number of cycles

Table 7: Overview of configurations implemented.

Table 7 summarises these configurations.

VARYING T. Since T translates to the number of squarings the client needs to perform to request an element, increasing it makes the puzzle more difficult. This allows for stronger deterrence depending on the use case and the size of the universe.

LINEAR SCALING. Since T is the number of squarings for requesting *one* element, T needs to scale linearly with the number of elements in a request. Otherwise an attacker can just request N elements in one request and only expend $\frac{T}{N}$ squarings.

EXPECTED CYCLES. This reflects the idea that the *expected* number of cycles a client needs to expend in order to find an element in $\mathcal{S}$

should be less than a certain threshold. A baseline T is computed by rounding to the next higher power of two for simplicity.

$$T_{\text{base}} = 2^{\log(\lfloor \frac{\mathbb{E}[c]}{\text{CYCLES_PER_SQUARING}} \rfloor + 1)}$$

where $\mathbb{E}[c]$ can be passed through the configuration. This requires an accurate measure of `CYCLES_PER_SQUARING`, which is discussed in Chapter 6.

For a request, the baseline T is scaled by the probability of a requested element being in $\mathcal{S}$ assuming uniform distribution:

$$T = T_{\text{base}} \cdot \mathbb{P}[\exists e \in \mathcal{E} : e \in \mathcal{S}] = T_{\text{base}} \cdot \frac{|\mathcal{S}|}{|\mathcal{U}|}$$

Remark. *This does leak information about the size of $\mathcal{S}$ at the time of the request, but depending on use case, $|\mathcal{S}|$ might not need to be secret.*

5.2 EVALUATION SETUP

For the following evaluations, we simulate multiple clients on different hardware. The extended hardware details and library versions are summarised in Table 8.

Component	Version/Details
Intel Machine	Intel Core i9-9900K CPU @ 3.6 GHz
→ OpenSSL	Version 3.0.16
→ mbedTLS	Version 3.6.0
→ Python	Version 3.13.2
Mac Book Air	Apple M2 @ 3.5 GHz
→ OpenSSL	Version 3.6.2
→ mbedTLS	Version 3.6.6
→ Python	Version 3.11.11

Table 8: Hardware details and library versions used for the evaluation.

The prototype implementation uses a squaring loop implemented using *mbedTLS*. Unless stated otherwise, it is the implementation used for the experiments, together with a 2048 bit modulus, a universe size and private set size of 10'000 and 100 respectively, and a single requested item without any further scaling of T .

For the comparison of different solvers, four methods across two machines are compared.

Namely, the same *mbedTLS*-based loop, a similar loop using the OpenSSL library, a single-call exponentiation via `BN_mod_exp` with exponent 2^T , as well as Python's built in `pow` function. All the Code Snippets can be found in the appendix in Section A.1.1.

Table 9 summarises how the terms used in the figures were measured.

Term	Measurement
Puzzle Generation	latency of <code>ecall_request_puzzle</code>
Puzzle Verification	latency of <code>ecall_submit_solution</code>
Puzzle Solver	latency of computing the solution of the puzzle

Table 9: Explanation of the measurement method for the terms used in the evaluation.

Remark. Every measurement was taken using the C++ *chrono* library. Note that each `ecall` includes a small constant overhead beyond the core operation. For example, `ecall_submit_solution` includes intersection computation in addition to puzzle verification. Since this overhead is constant across all measurements, it does not affect the scaling behaviour, which is what the following analysis relies on.

EVALUATION

6.1 MEASUREMENTS

The following data was collected using the evaluation harness described in Section 5.2. The source code is available in the Repository mentioned in Section A.1 in the Appendix.

6.1.1 Linear Scaling in T

Figure 8 suggests that the time for solving a puzzle scales linearly in T .

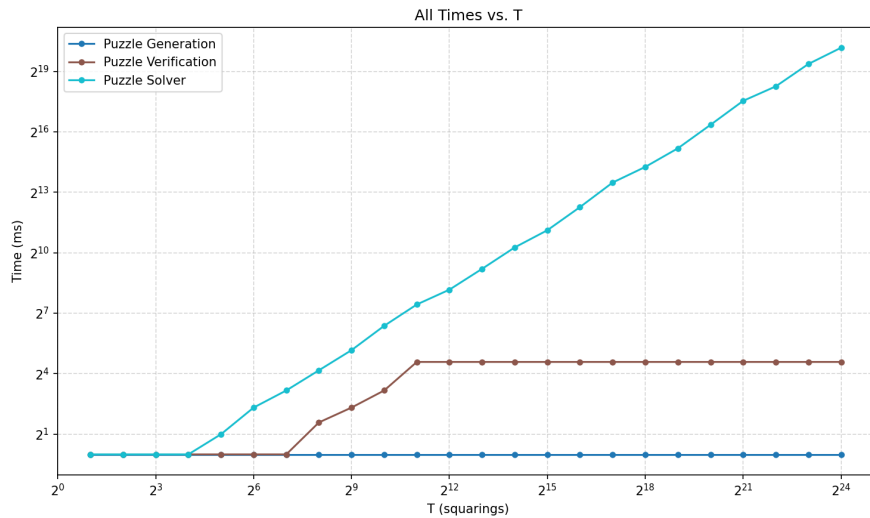


Figure 8: Fixing all other configurations, the time to solve a puzzle scales linearly in T , while generating and verifying a puzzle stays constant.

Importantly however, both the time it takes to generate a puzzle and to verify a solution do not scale meaningfully with T . This confirms the shortcut using the secret ϕ in order to precompute the exponent.

With the loop-based implementation in mind, this is to be expected, as an increase in T translates directly to an increase in the number of iterations. As Section 6.1.3 will suggest, this scaling behaviour is common amongst all evaluated solvers and confirms the theoretical assumption of T sequential squarings being necessary.

6.1.2 Additional Measurements

Experiments also confirmed that the configurations seem to have the expected impact on the solve time and that the crypto initialisation scales exponentially in the number of bits used for the modulus.

The Figures showing various experimental results can be found in the Appendix in Section A.2.

6.1.3 Required Cycles

In order to make realistic parameter choices for this TLP-based rate-limiting implementation, it is crucial to examine the computing resources used by different solvers.

MBEDTLS IMPLEMENTATION. As already discussed, the *mbedtls* based implementation used for the previous evaluations results in linear scaling of computation time with T . The reason for that is, that every iteration uses a roughly constant number of cycles. Figure 9 shows whether the number of cycles for one squaring depends on T or the iteration of the loop. It suggests that the number of cycles remains roughly constant in both T and iteration.

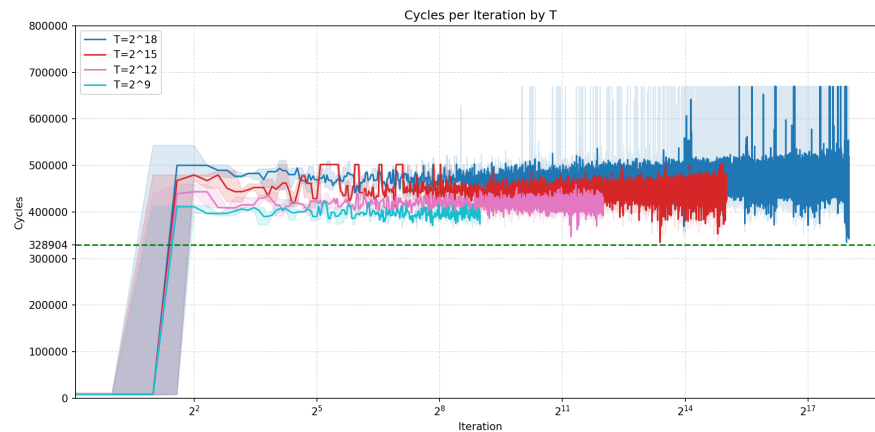


Figure 9: The number of cycles measured in each iteration of a squaring loop for multiple T . The required cycles are largely independent of the iteration and T , allowing us to set a lowerbound.

Remark. *The largest 1% of them have been cut off, as they reflect external factors like context switches or interrupts and are thus not representative of the actual squaring implementation.*

The fact that the number of cycles do not decrease drastically after the first few iterations allows us to set a conservative lower bound at 300'000 cycles per squaring across all iterations.

OTHER IMPLEMENTATIONS. In order to make general claims, it is necessary to consider different implementations.

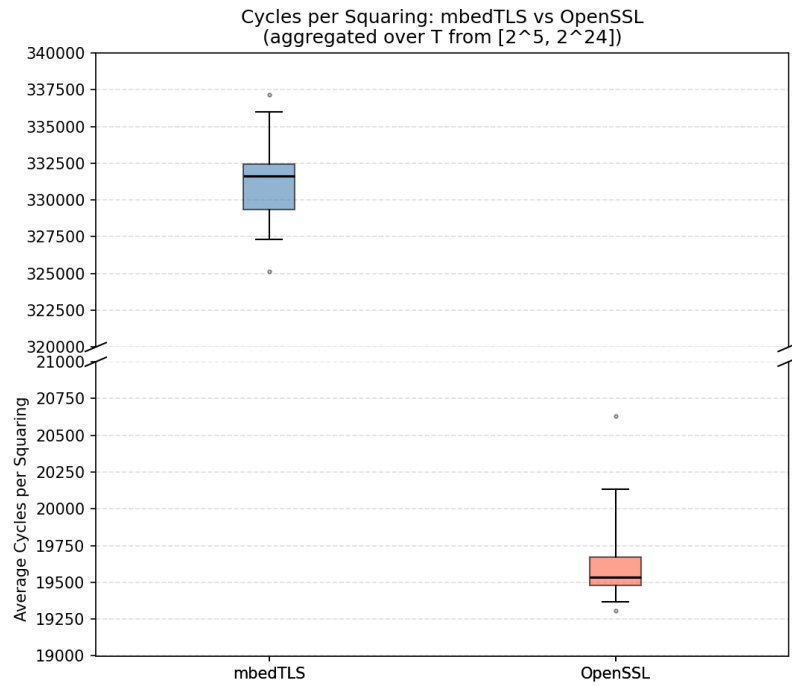


Figure 10: The average cycles used per squaring iteration by the *OpenSSL-loop* implementation compared to *mbedTLS* with different T .

Figure 10 compares the average cycles per squaring of *mbedTLS* loop with the *OpenSSL-loop* implementation. While *OpenSSL* shows a similarly small variance with respect to T , the absolute value is approximately 17 times lower, making *OpenSSL* much more efficient.

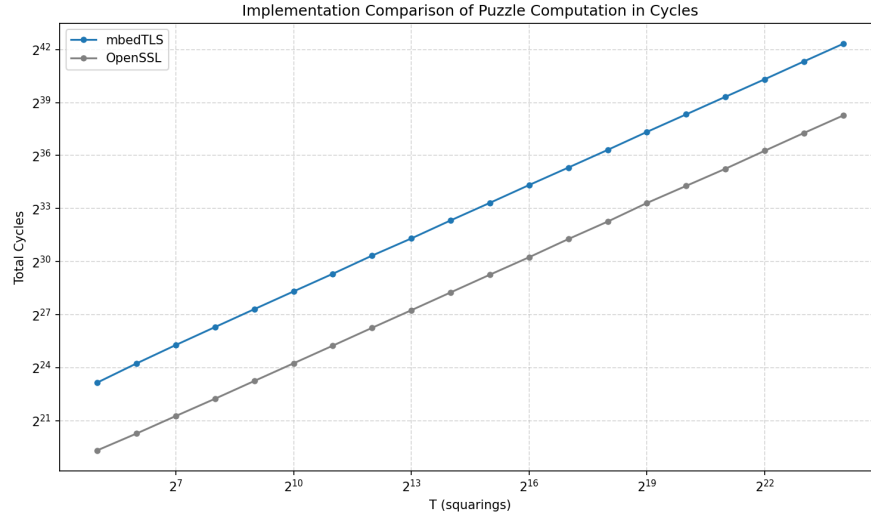


Figure 11: The total number of cycles required by *mbedTLS* and *OpenSSL-loop* for increasing T : The *OpenSSL-loop* implementation is considerably more efficient, but both are linear in T

Importantly, Figure 11 shows that even though *OpenSSL-loop* is much more efficient, the number of cycles for solving a puzzles still scales linearly in T .

6.1.4 Comparing Wall-Clock Time

To get a more practical view of the computational effort required for different implementations, we can look at the wall-clock time that different solvers use on different hardware. Figure 12 compares all implementations and hardware. The linear dependence on T is present throughout all of them, but different implementations have a big effect on the multiplicative factor. The *mbedTLS* solver on Intel is the slowest, while *OpenSSL-exp* is the fastest, being roughly 128 times quicker. The drastic increase in latency for the Apple M2 hardware after roughly 8 seconds is most likely due to CPU throttling.

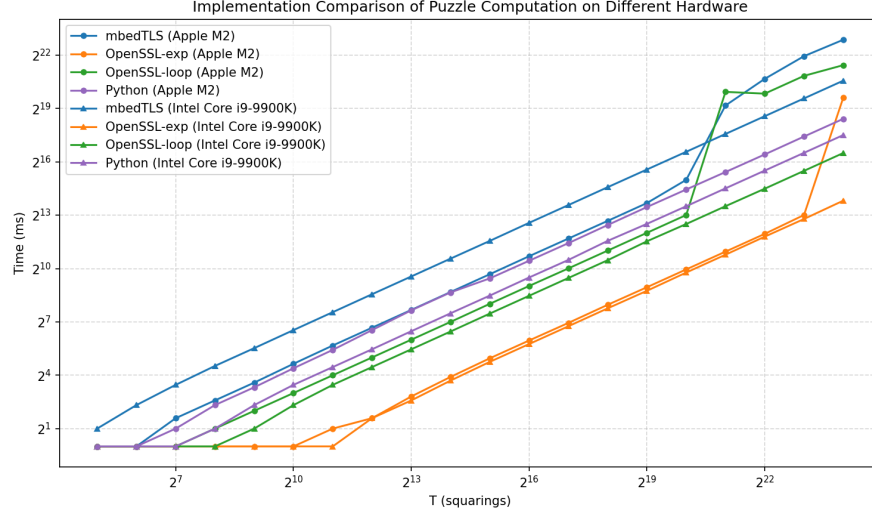


Figure 12: Comparison of the latency of different solvers on different hardware. While they are all linear in T , their efficiency varies drastically.

6.2 SECURITY ANALYSIS

The guarantees of [TEEs](#) are assumed to hold and the analysis is strictly limited to any effects *introduced* by the rate-limiting.

6.2.1 Security Properties of PSI

We now analyse the security of the implementation against the defined properties of [PSI](#).

PROPERTY 1. Firstly, since private set is contained in the Enclave, it is protected from any attacker. The guarantees of the [TEE](#) satisfy confidentiality of both memory and execution, thus fulfilling [P1](#).

PROPERTY 2. Secondly, the rate-limiting does not depend on the private set at all. In particular, the only interaction with it is through the controlled `compute_intersection` interface. Since the rate-limiting only forwards its result, [P2](#) is also satisfied.

PROPERTY 3. Finally, the implementation only returns a result from the service, if the submitted solution and the correct solution are equal.

As discussed in [Section 4.5](#), this must mean that the required sequential squarings were performed.

As the experimental results suggest, we can set a lower bound on the number of cycles needed per squaring for a *given implementation*. This supports the claim that the computational cost ($\mathbf{W}_{\min}, \mathbf{D}_{\min}$) has been expended.

By defining T sufficiently large to suit our needs and according to hardware assumptions, we can thus satisfy **P3**.

However, the experiments also showed that different implementations widely differ in their required cycles. The *OpenSSL-loop* solver for example is roughly 17 times more efficient, while *OpenSSL-exp* is another 8 times faster. Specialised implementations can likely achieve additional speedups, reinforcing the lack of concrete lower-bound guarantees.

More precisely, for a given implementation requiring at least c cycles per squaring on hardware running at frequency f Hz, the wall-clock lower bound on solving the puzzle is $T \cdot \frac{c}{f}$ seconds. As the experimental results in Section 6.1.3 suggest, $c \geq 300'000$ cycles for the *mbedtls* implementation. However, as the comparison with OpenSSL demonstrates, a more efficient solver reduces this bound significantly, but choosing T conservatively with another buffer factor further punishes legitimate clients without specialised solvers.

6.2.2 Attacks

TIME MANIPULATION. Since the trusted part of the implementation does not depend on any notion of time, it is not vulnerable to any time manipulation attacks. Thus, there is no need for a **T2** time source.

ROLLBACK. The implementation does not hold any accumulating state, since it is stateless by design. The only persistent objects are configuration parameters and the exponent. The configuration is assumed to be constant and the exponent is adjusted deterministically for every request, so rolling it back gains no advantage. It thus inherits the rollback security discussed in Section 4.5.

PARALLEL REQUESTS. The experimental results confirm that linear scaling with the input size effectively enforces the per-element bound over a batch of elements. Unlike multiple single requests, a large one cannot be parallelised.

However, an attacker is not forced to submit large requests. By splitting a batch of N elements into N single-element requests, the total work W remains identical, but the depth D is reduced by a factor of N , since all N puzzles can be solved in parallel.

Possible mitigations like enforcing a minimal batch size or punishing small requests are impractical, since they increase the cost of legitimate clients drastically.

A complete solution would be to only allow one open puzzle per client at a time, but as discussed in Section 4.5, this is not possible in a stateless design.

This once again reinforces that the lower bound on $\mathbf{D}$ should be understood as a per-element guarantee, and any definition of *sufficiently costly* that relies purely on depth must account for this.

CLONING. Clones do not reduce the amount of work being done, so it has no effect on $\mathbf{W}$.

The throughput of a parallel attacker solving and submitting many puzzles simultaneously will at some point be limited by the sequential latency of `ecall_request_puzzle` and `ecall_submit_solution`. Cloning can remove this bottleneck, allowing an adversary to process arbitrarily many parallel puzzle submissions simultaneously, further reducing the amortised $\mathbf{D}$ cost of full enumeration.

TIMING SIDE CHANNEL ATTACKS. The existence of single-stepping frameworks for [TEEs](#), which allow the untrusted host to execute single instructions of the enclave code precisely, makes timing side-channel a legitimate concern [12]. By timing single instructions, even small variances can be detected with high accuracy.

The main attack surface for timing attacks is the comparison of the correct Y_{true} and the user-supplied Y . Conventional comparisons, like the built in `mbedtls_mpi_cmp_mpi`, are not constant time since they exit as soon as the first position differs. This allows for an attack where the attacker submits arbitrary Y values without solving the puzzle, measures response timing, and uses early-exit behaviour to determine how many leading "chunks" of Y_{true} their guess matches, building up the correct solution byte by byte. This costs the attacker essentially nothing computationally, since they never need to perform squarings violating $\mathbf{P}_3$.

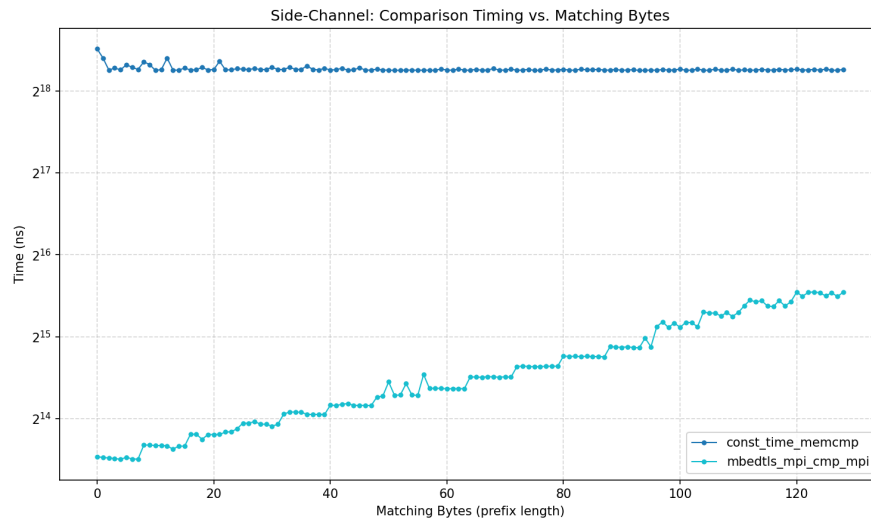


Figure 13: Comparison of constant time comparison and the built in `mpi` comparison, which leaks the length of the matching prefix.

The solution is using a custom constant time comparison like `const_time_memcmp` that compares padded buffers without exiting early. Figure 13 shows how the `mpi` comparison leaks information about the length of the matching prefix, while the custom comparison stays constant.

Another vulnerability is the exponentiation, which theoretically has latency dependent on the exponent. Thus computing $y_{\text{true}} = x^e \pmod{N}$ leaks timing information about the exponent. If an attacker would learn e , it could bypass the T squarings completely.

Library methods of *MBEDTLS* are protected against known software attacks, but makes no claims regarding an attacker with hardware access [28].

This makes it insufficient for the assumed threat model in this thesis, where the host has full control over the machine.

NON-TIMING SIDE CHANNELS. Similarly, *MBEDTLS* does not provide any guarantees against non-timing side channels [28]. An attacker can replay operations containing ϕ or e arbitrarily often in order to try and learn them for example from their cache access pattern. If they are learned, any puzzle using the same modulus can be computed efficiently by the attacker. However, mitigating non-timing side channels is out of scope for this thesis.

6.3 ADVERSARY INSTANTIATIONS

In the following, we illustrate the previous arguments in some concrete examples, each assuming a different adversarial model. Adversarial models are differentiated on three domains: implementation efficiency (cycles/squaring), clock frequency (cycles/ns) and parallel resources. Their respective values are summarised in Table 10

6.3.1 Adversary Models

Adversary	Implementation	Frequency	Parallel
A1: Standard	300 k	3.6 GHz	1
A2: Optimised	2 k	6 GHz	1
A3: Parallel	30 k	3.6 GHz	∞

Table 10: Instantiations of adversarial models.

For the following examples, we make some simplifying assumptions, namely that the cycles for a squaring are done sequentially and that these are the only factors influencing the total computation work and time.

A1: STANDARD For a standard client, we assume an implementation using *mbedtls* which results in the 300'000 cycles per squaring. The hardware is assumed to run on 3.6 GHz, which corresponds roughly to the *mbedtls* solver used in Section 6.1.3.

The total number of cycles for full enumeration for this attacker is thus $\mathbf{W}_1 \geq |\mathcal{U}| \cdot 300'000 \cdot T$. Since we assume no parallelisation, the sequential number of cycles is the same: $\mathbf{D}_1 = |\mathcal{U}| \cdot T$. We can derive a lower bound on wall-clock time of $\tau_1 \geq \frac{\text{cycles}}{\text{frequency}} = \frac{|\mathcal{U}| \cdot T \cdot 300'000}{3.6 \cdot 10^9}$ seconds.

To satisfy **P3**, choose

$$T \geq \frac{\mathbf{W}_{1\min}}{|\mathcal{U}| \cdot 300'000}$$

where $\mathbf{W}_{1\min}$ is the target minimum cost. Alternatively, we choose

$$T \geq \frac{\tau_{1\min} \cdot 3.6 \cdot 10^9}{|\mathcal{U}| \cdot 300'000}$$

where $\tau_{1\min}$ is the target minimum wall-clock time.

A2: OPTIMISED For an optimised attacker, we assume an *OpenSSL-exp* solver. According to Figure 10 conservative lower bound for the cycles per squaring of *OpenSSL-loop* can be set at roughly 17'000. Figure 12 shows that *OpenSSL-exp* is roughly 8 times faster than that, so we assume 2 k cycles per squaring. We additionally assume a specialised high-frequency hardware running at 6 GHz.

The total number of cycles for full enumeration for this attacker is now $\mathbf{W}_2 \geq |\mathcal{U}| \cdot 2'000 \cdot T$. Again, assuming no parallelism means the sequential guarantee is $\mathbf{D}_2 = |\mathcal{U}| \cdot T$. With the faster clock frequency, we get a newer wall-clock bound of $\tau_2 \geq \frac{|\mathcal{U}| \cdot T \cdot 2'000}{6 \cdot 10^9}$ seconds, which is 250 times lower ($\frac{300'000/3.6}{2'000/6} = 250$).

To satisfy **P3**, we have to choose

$$T \geq \frac{\mathbf{W}_{2\min}}{|\mathcal{U}| \cdot 2'000}$$

where $\mathbf{W}_{2\min}$ is the target minimum cost. Alternatively, we can choose

$$T \geq \frac{\tau_{2\min} \cdot 6 \cdot 10^9}{|\mathcal{U}| \cdot 2'000}$$

where $\tau_{2\min}$ is the target minimum wall-clock time.

For $\mathbf{W}_{2\min}$, we have to choose T roughly 150 times higher, while it needs to be 250 times higher for $\tau_{2\min}$ compared to **A1**.

A3: PARALLEL For the highly parallel attacker, we assume an *Python* implementation and a clock frequency of 3.6GHz. Figure 12 indicates that the *Python* solver is roughly 2 times slower than *OpenSSL-loop* on average. We use 30 k for this example.

However, we assume the attacker has infinite parallel hardware, which they use to request $|\mathcal{U}|$ single elements in parallel. To remove any potential bottlenecks from puzzle generation and verification by the enclave, we assume infinite Enclave clones.

Remark. *In practice, there is a physical limit to the number of solvers and clones that can run in parallel, defined by the available cores. This models the theoretical worst case for the **D** guarantee rather than a realistic adversary.*

The total number of cycles for full enumeration for this attacker is still $\mathbf{W}_3 \geq |\mathcal{U}| \cdot 30'000 \cdot T$. The parallelisation does not reduce them.

However, the new sequential guarantee is reduced to $\mathbf{D}_3 = T$.

The only possible wall clock bound is $\tau_3 \geq \frac{T \cdot 30'000}{3.6 \cdot 10^9}$ which is constant in $\mathcal{U}$.

To satisfy **P3**, we can only choose

$$T \geq \frac{\mathbf{W}_{3\min}}{|\mathcal{U}| \cdot 30'000}$$

where $\mathbf{W}_{3\min}$ is the target minimum cost. Since the sequential guarantee does not scale, we cannot reasonably satisfy **P3** through $\mathbf{D}_3$.

6.3.2 Practical Examples of Private Sets

To give a sense of what these numbers mean in practice, consider the following illustrations.

A1 WITH USERNAMES. We hold a private set consisting of user names of our application, and we assume that usernames are consisting of exactly 5 lowercase latin letters. The universe size is $|\mathcal{U}| = 26^5$. We assume attacker **A1**.

If we want full enumeration of this universe to take at least *a year* ($\tau_{1\min} \geq 31'536'000$ seconds), we can compute a minimum required

$$T \geq \frac{\tau_{1\min} \cdot 3.6 \cdot 10^9}{|\mathcal{U}| \cdot 300'000} = \frac{31'536'000 \cdot 3.6 \cdot 10^9}{26^5 \cdot 300'000} = 31'850 \approx 2^{15}$$

A1 represents the client used for the evaluation data. Figure 8 confirms these calculations empirically. The full enumeration bound results in a per-element bound of $\frac{\tau_{1\min}}{|\mathcal{U}|} = \frac{31'536'000}{11'881'376} = 2.654...$ seconds ($\approx 2^{11}$ milliseconds), which is also achieved by $T = 2^{15}$ according to the figure.

A2: PASSWORD PINS. We now assume that our private set consists of 4-digit pins, with the size of the universe being $|\mathcal{U}| = 10^4$. Additionally, we assume attacker **A2**.

We want full enumeration of this universe to take at least *a week* ($\tau_{2\min} \geq 604'800$ seconds).

This time, we need a much larger T , to satisfy this bound:

$$T \geq \frac{\tau_{2\min} \cdot 6 \cdot 10^9}{|\mathcal{U}| \cdot 2'000} = \frac{604'800 \cdot 6 \cdot 10^9}{10^4 \cdot 2'000} = 181'440'000 \approx 2^{28}$$

The required full enumeration time results in a per-element cost of $\frac{\tau_{2\min}}{|\mathcal{U}|} = 60.4$ seconds. This is much more than can reasonably be asked from a legitimate client. This issue is exaggerated further, if the client is less efficient. Extrapolating the required $T = 2^{28}$ with the *mbedTLS* solver from Figure 12 shows that this will result in multiple hours. This is a direct result of the low entropy of the set, a limitation that is discussed further in Section 7.1.

A3: PHONE NUMBERS. Finally, we hold a private set consisting of phone numbers of the format '+41 xxx xxx xx xx'. The universe size is $|\mathcal{U}| = 10^{10}$ and we consider attacker **A3**. We want to choose $W_{3\min}$ such that the economic cost of running a full enumeration on an on-demand AWS VM is in the order of **\$100'000**. According to current dynamic pricing, the cost is roughly \$0.05 per vCPU-Hour [5]. The \$100'000 buys $\frac{\$100'000}{\$0.05} \cdot 3600 = 7.2 \cdot 10^9$ seconds of compute. Assuming 3.6 GHz clock frequency results in $W_{3\min} = 7.2 \cdot 10^9 \cdot 3.6 \cdot 10^9 \approx 2.6 \cdot 10^{19}$ cycles. Choosing

$$T \geq \frac{W_{3\min}}{|\mathcal{U}| \cdot 30'000} = \frac{2.6 \cdot 10^{19}}{10^{10} \cdot 30'000} = 86'666.666... \approx 2^{17}$$

is sufficient.

Even though a brute-force attack would cost a highly parallel attacker \$100'000 when choosing $T = 2^{17}$, making a very strong guarantee on W , we still cannot make a better wall-clock time guarantee than $\tau_{3\min} \geq \frac{T \cdot 30'000}{3.6 \cdot 10^9} \approx 1$ second. A parallel attacker highlights that the **D** dimension is a per-element guarantee.

6.3.3 Summary of Adversarial Models and Examples

Example	Attacker	$ \mathcal{U} $	Target	T	Latency
Usernames	A1	26^5	1 year	2^{15}	2.654 s/elem
PINs	A2	10^4	1 week	2^{28}	60.4 s/elem
Phone numbers	A3	10^{10}	\$100 k	2^{17}	1.092 s total

Table 11: Summary of private set examples and derived parameters.

Table 11 summarises the three examples and their derived parameters.

The username example with attacker **A1** showed that the theoretical bounds closely aligns with the observed experimental data, when hardware assumptions are similar.

The examples also illustrate that the appropriate notion of *sufficiently costly* depends heavily on both the universe size and the assumed attacker. For large, high-entropy universes like phone numbers with an average attacker, $T = 2^{17}$ is enough to impose a significant economic cost on full enumeration, even if the attacker is highly parallel. For small, low-entropy universes like 4-digit PINs, the required T grows to the point where legitimate clients have to pay an unreasonable cost.

Additionally, the phone number example demonstrates that while **P3** can be enforced through constraints on **W**, the **D**-guarantee collapses to a single element bound under parallelism, reducing full enumeration to one second regardless of universe size.

DISCUSSION

This chapter reflects on the limitations of the proposed design, identifies directions for future work, and summarises the contributions of this thesis.

7.1 LIMITATIONS

The following are limitations of the proposed approach that are not addressed by the current design. They represent inherent consequences of a stateless, [TLP](#)-based approach.

NO LOWER BOUND GUARANTEES. While the experimental results support the argument, that solving the [TLP](#) results in a predictable cost in terms of cycles, it is impossible to set a non-trivial rigorous lower bound. Any guarantee is always dependent on the considered hardware and implementation. It is always possible that a faster implementation comes along and further reduces the multiplicative constant of required cycles per squaring. Just like *OpenSSL-exp* is roughly 128 times faster than *mbedTLS* according to Figure 12, there might be other implementations gaining a similar speedup.

Similarly, it is not possible to convert the per-puzzle cycle guarantees into a lower bound on wall-clock time. This is due to the possible parallelisation across multiple puzzles and the conversion of cycles into wall-clock time being hardware dependent. Using specialised hardware such as FPGAs or ASICs could increase efficiency further.

OVERHEAD FOR LEGITIMATE CLIENTS. As the examples in Section 6.3.2 showed, for private sets with low entropy, e. g. small universes, the rate-limiting has to be very aggressive. A universe of size 10'000, for example cannot practically be protected using rate-limiting, because it makes single requests unreasonably expensive. This is an inherent problem with any sort of stateless brute-force protection. The reason is that the fraction of elements a legitimate client will query is much greater, so the same cost or delay on full enumeration will result in much greater relative punishment to the client.

To combat this, the rate-limiter would need to store state in order to increase cost exponentially per client, similarly to how mobile phones do for passcodes.

COST ACCUMULATES OVER TIME. Unlike delay-based rate-limiting, where small per-request delays become negligible over a long time-

line, TLP-based rate-limiting imposes a fixed computational cost per request regardless of when it is made. A legitimate client making many requests over months or years accumulates the same total cost as an attacker attempting brute-force enumeration. This is an inherent consequence of the cost-per-element model and may be unacceptable in settings where legitimate clients are expected to query a significant fraction of the universe over time. Consider the phone number example in Section 6.3.2, where a legitimate client that queries half of the universe over a decade would still be paying \$50 k in compute costs due to the rate-limiting.

NEW PROTOCOL. Finally, adaptation of the implemented rate-limiting approach requires a change in protocol. The two-phase protocol introduces inconvenience to legitimate clients, which have to program against this API. This stands in contrast to a traditional rate-limiting primitive, which can easily be plugged into an existing interface without any change for the clients.

7.2 FUTURE WORK

Several directions remain open for extending the work presented in this thesis. Examples include the following.

SIDE CHANNEL VULNERABILITIES. The most direct extension to the work presented in this thesis is hardening the implementation against side channel vulnerabilities. This could include using a different crypto library that makes stronger side channel guarantees and testing puzzle generation and verification for other leaks.

DISTRIBUTION OF THE UNIVERSE. The current probability-based configuration uses a very simple model: The universe size is derived directly from the degrees of freedom given by the format of the elements in $\mathcal{S}$. Depending on semantics of the set, this could be extended to model the probability of a requested element being in the private set.

One has to be careful not to leak any information about the contents of $\mathcal{S}$ though, so assuming anything other than $\mathcal{S}$ being drawn uniformly at random from $\mathcal{U}$ is invalid.

However, the "semantic size" of the universe could potentially be derived more carefully. As an example, let us revisit swiss phone numbers of the format '+41 xxx xxx xx xx'. With some domain-specific knowledge, we can observe that the first three digits represent the provider, of which there are only a few big ones. As a consequence, most of swiss phone numbers start with either '076', '078' or '079'. The actual universe size might thus be closer to $3 \cdot 10^7$ than 10^{10} and T should consequently be chosen larger.

BROADER APPLICATIONS. The TLP-based primitive is not specific to the PSI setting. Any TEE interface where repeated queries leak sensitive information is a candidate. Examples include access control interfaces, oblivious lookup structures, or interfaces susceptible to side-channel attacks. The specific design choices and configuration decisions would depend on the semantics of the interface and the information exposed.

AUTOMATIC CONFIGURATION. The current implementation requires the configuration to be chosen manually. After that, it is applied equally to every request. As discussed, this forces it to be conservative.

A more fine-grained approach would be to decide the difficulty parameters based on trusted information about the resources of each client.

7.3 CHOOSING A RATE-LIMITING PRIMITIVE

This thesis explored different approaches to rate-limiting a PSI interface in an untrusted host environment.

Based on the experimental data, we conclude that TLP-based approaches can enforce real costs on clients, even against capable attackers, making brute-force attacks very hard. They work particularly well when the universe is very large and legitimate clients only ever query small subsets of it. Under these assumptions, we can make the puzzles difficult enough to deter even very efficient adversaries, while still keeping the cost to the average client small. However, as mentioned throughout, if some of the assumptions change, other primitives become more appropriate.

For example, if the TEE environment in which the PSI service is running provides additional hardware primitives like a T₂ time source and MCs, then Algorithm 1 can be enforced more realistically and makes stronger wall-clock guarantees. Such a design remains much more usable to the average client. While the attack surface is not entirely closed, remaining attacks like cloning require considerable skills and know-how, so leaving them unaddressed might be a reasonable tradeoff.

Furthermore, if the private set is deployed for a host that is simultaneously the sole client and bears the computational cost, a busy loop is simpler and can give slightly stronger guarantees, since the implementation details are fixed. This could be the case for an interface that leaks information to the host unintentionally and rate-limiting is used to limit side-channel vulnerabilities.

Additionally, for low-entropy universes no stateless rate-limiting can simultaneously enforce great cost on full enumeration while still keeping individual queries practical. In such a scenario, it might be

necessary to implement additional primitives like [MCs](#) because keeping per-client state to enforce exponential difficulty is unavoidable.

Finally, as discussed in Section [7.1](#), in a setting where legitimate clients are expected to approach full enumeration over a long timeline, the resulting computational cost might simply be unacceptable, making a delay-based approach a more suitable choice, even if certain attacks remain feasible.

7.4 CONCLUSION

This thesis explored the [PSI](#) problem in an environment, where the host of the service is untrusted. In this setting, we need [TEEs](#) in order to enforce a rate-limit on the interface. Otherwise, the private set can be learned by full enumeration of the universe of $\mathcal{S}$. Implementing this in a [TEE](#) environment is non-trivial. This thesis discussed three main attacks that rate-limiting needs to be resistant to. The lack of a trusted time source, rollback attacks and cloning undermine any guarantees and additional frameworks that address these attacks add complexity.

An alternative approach is using computational puzzles to make requests costly. They allow for a stateless design that places the rate-limiting burden on the client, which makes it resistant to the mentioned attacks. From the explored primitives, [TLPs](#) were chosen, because they can make guarantees both in terms of total required cycles, as well as necessary sequential steps.

A prototype implementation in Intel SGX was presented and the evaluated experimental data supports the theoretical guarantees that [TLPs](#) provide.

The results suggest that [TLPs](#) are self-contained alternative to counter-based rate-limiting in [TEEs](#), avoiding the additional primitives required by the strawman approaches in Chapter [3](#) and only increasing the [TCB](#) slightly.

Fundamental limitations exist however, including the lack of precise guarantees and considerable cost to legitimate clients, making their practical use limited to specific use cases.

APPENDIX

A.1 SOURCE CODE

The full source code for both the Intel SGX-SDK implementation, as well as a proof-of-concept for a VM-level TEE is available at:

https://github.com/moritzMantel/Bachelor_Thesis_ETH

Commit: 2c8624c

A.1.1 Solver Implementations

The following Code Snippets show the solvers used in the evaluation.

```
// T      <- number or squarings
// N_mpi   <- MPI repr of modulus (N)
// x_mpi   <- MPI repr of base (x)
for (uint64_t i = 0; i < T; i++) {
    mbedtls_mpi_mul_mpi(&x_mpi, &x_mpi, &x_mpi);    // x = x * x
    mbedtls_mpi_mod_mpi(&x_mpi, &x_mpi, &N_mpi);    // x = x % N
}
// x_mpi now contains the solution x^2^T mod N
```

Code Snippet 1: Solver implementation: *mbedtls*

```
// T      <- number or squarings
// N_BN    <- BIGNUM repr of modulus (N)
// x_BN    <- BIGNUM repr of base (x)
for (uint64_t i = 0; i < T; i++) {
    BN_mul(x_BN, x_BN, x_BN, ctx);    // x = x * x
    BN_mod(x_BN, x_BN, N_BN, ctx);    // x = x % N
}
// x_BN now contains the solution x^2^T mod N
```

Code Snippet 2: Solver implementation: *OpenSSL-loop*


```

// T_BN      <- BIGNUM repr of T
// two_BN    <- BIGNUM repr of 2
// N_BN      <- BIGNUM repr of modulus (N)

BN_exp(T_BN, two_BN, T_BN, ctx);           // 2^T
BN_mod_exp(x_BN, x_BN, T_BN, N_BN, ctx);   // x = x^2^T mod n

// x_BN now contains the solution x^2^T mod N

```

Code Snippet 3: Solver implementation: *OpenSSL-exp*

```

# T      <- number of squarings
# N      <- modulus (N)

y = pow(x, 2**T, N)

# y now contains the solution x^2^T mod N

```

Code Snippet 4: Solver implementation: python

A.2 FURTHER EVALUATION FIGURES

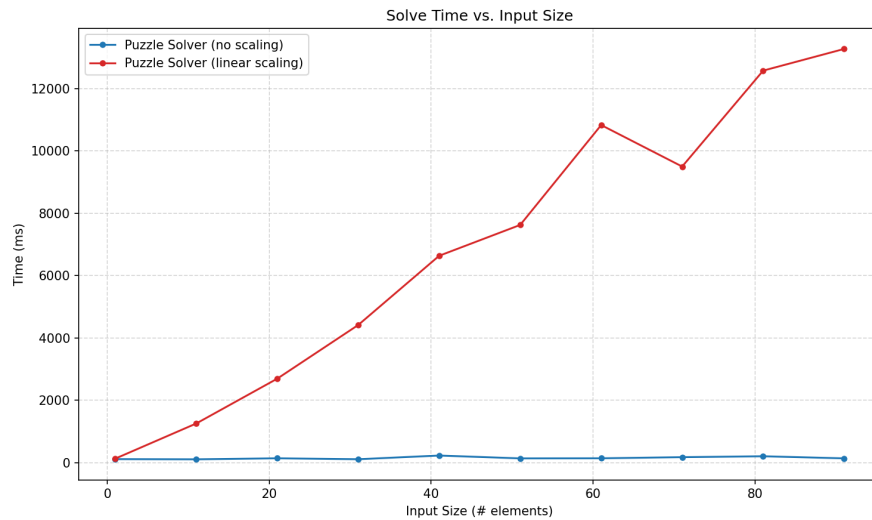


Figure 14: Scaling of solve time with input size: with T and all other configurations fixed, linear scaling in input size has the expected linear consequence for the solver time.

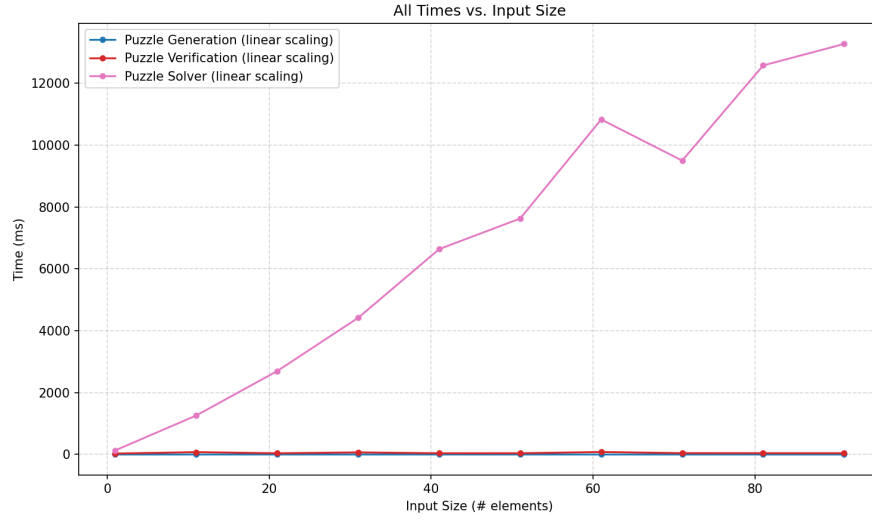


Figure 15: Scaling of solve time with input size: with T and all other configurations fixed, the linear scaling does not influence generation or verification time.

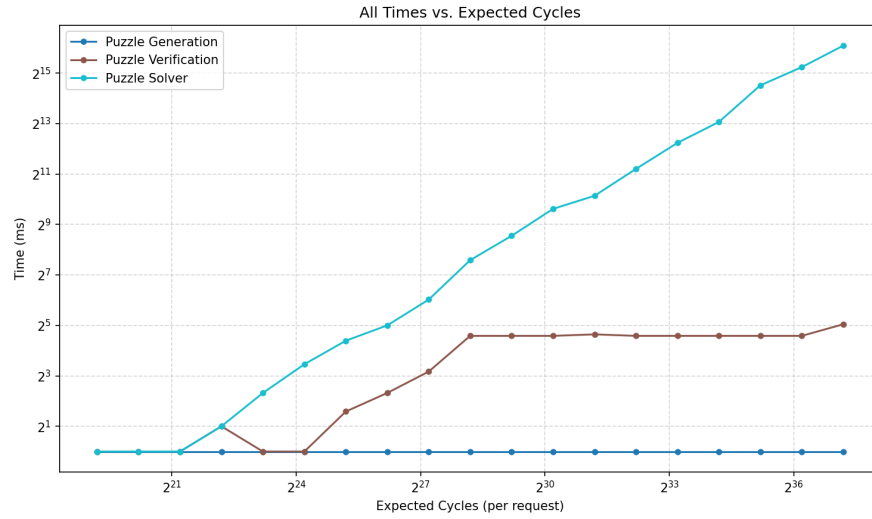


Figure 16: Configuring $T_{\text{baseline_comp}}=1$, T is computed such that the expected number of cycles to solve the puzzle is at least the **Expected Cycles**. As a reference, 300'000 cycles per iteration was used.

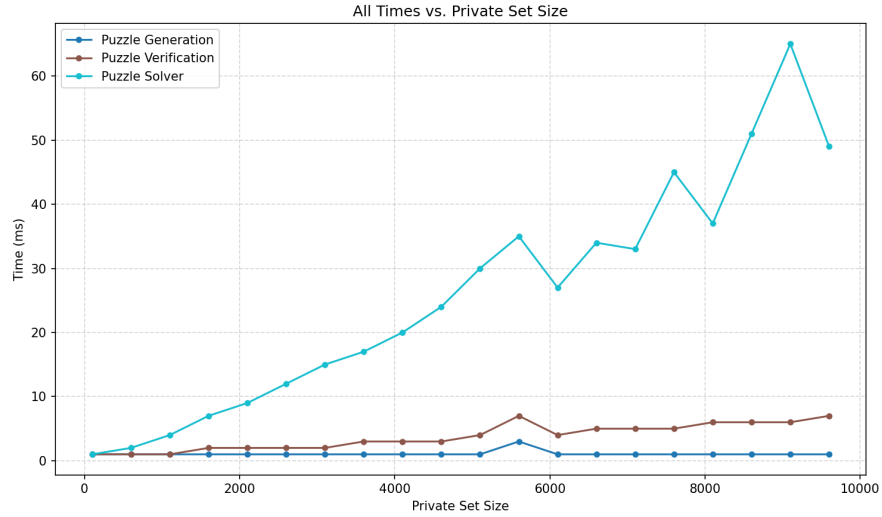


Figure 17: Configuring $T_{\text{baseline_comp}}=1$ and $T_{\text{private_set_comp}}=1$, T is computed such that the expected number of cycles to hit an element in S when randomly probing is at least **Expected Cycles**. As a reference, 300'000 cycles per iteration was used. The solve time scales linearly in $\frac{|S|}{|U|}$. The fluctuations might be explained through rounding errors.

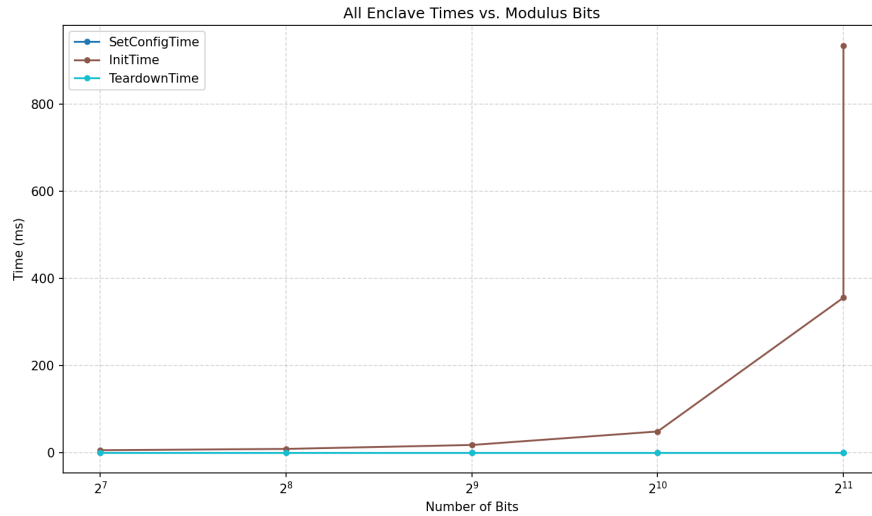


Figure 18: The latency of initialising the crypto primitives scales exponentially in the number of bits used for the modulus. The overhead of ecalls is in the order of a few milliseconds.

A.3 USE OF GENERATIVE AI

The table on the next page summarises how and where generative AI was used for this thesis.

Tool	Purpose	Limitation	Affected Work
Claude	Language and formatting assistance (style, grammar, notation consistency, LaTeX syntax)	Proposed changes critically evaluated, only accepted changes regarding language, formatting implemented manually	entire thesis
	sample "skeleton" text for structure	no full text copied, only structure applied	Abstract
	generate feedback on flow and gaps in arguments	feedback critically evaluated and improvements applied manually	entire thesis
	generate feedback on code quality and correctness	changes implemented manually	Source code for TLP_VM_TEE/ and TLP_INTEL_SGX/
	generated centralised and cleaned up scripts	individual code written independently, only cleaned up and put together	plot_experiments.py, python_experiments.py
NotebookLM	propose further sources	sources read and used manually	Ch. 2, 4
	categorise and condense information from multiple sources for diverse citations	sources read and understood independently, truthfulness of citations checked	Ch. 2
	generate feedback	feedback analysed critically, changes applied manually	entire thesis
ChatGPT	generate feedback	feedback analysed critically, changes applied manually	entire thesis

Table 12: Overview of generative AI usage.

BIBLIOGRAPHY

- [1] AMD Technical Information Portal. URL: https://docs.amd.com/v/u/en-US/40332-PUB_4.08 (visited on 03/02/2026).
- [2] AMD Technical Information Portal. URL: <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more> (visited on 06/03/2026).
- [3] ARM Security Technology Building a Secure System using TrustZone Technology. URL: <https://developer.arm.com/documentation/PRD29-GENC-009492/c/TrustZone-System-Design/Gadget2008-specification/Content-management-specification?lang=en> (visited on 03/02/2026).
- [4] Fritz Alder, Gianluca Scopelliti, Jo Van Bulck, and Jan Tobias Mühlberg. "About Time: On the Challenges of Temporal Guarantees in Untrusted Environments." In: *Proceedings of the 6th Workshop on System Software for Trusted Execution*. SysTEX '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 27–33. ISBN: 979-8-4007-0087-3. DOI: [10.1145/3578359.3593038](https://doi.org/10.1145/3578359.3593038). URL: <https://dl.acm.org/doi/10.1145/3578359.3593038> (visited on 03/02/2026).
- [5] Amazon EC2 Instance Comparison. URL: <https://instances.vantage.sh/?id=7cdfa91574f14c2f5c112f35c6a43f36ec121f05> (visited on 06/15/2026).
- [6] Fatima M. Anwar and Mani Srivastava. "Applications and Challenges in Securing Time." In: *12th USENIX Workshop on Cyber Security Experimentation and Test (CSET 19)*. Santa Clara, CA: USENIX Association, Aug. 2019. URL: <https://www.usenix.org/conference/cset19/presentation/anwar>.
- [7] Alexey V. Bataev, Dmitriy G. Rodionov, and Darya A. Andreyeva. "Analysis of world trends in the field of cloud technology." In: *2018 International Conference on Information Networking (ICOIN)*. 2018, pp. 594–598. DOI: [10.1109/ICOIN.2018.8343188](https://doi.org/10.1109/ICOIN.2018.8343188). URL: <https://ieeexplore.ieee.org/abstract/document/8343188>.
- [8] Guy E. Blelloch. "Programming parallel algorithms." In: *Commun. ACM* 39.3 (1996), pp. 85–97. ISSN: 0001-0782. DOI: [10.1145/227234.227246](https://doi.org/10.1145/227234.227246). URL: <https://dl.acm.org/doi/10.1145/227234.227246> (visited on 06/01/2026).
- [9] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. "Verifiable Delay Functions." In: *Advances in Cryptology – CRYPTO 2018*. Cham: Springer International Publishing, 2018, pp. 757–788. ISBN: 978-3-319-96884-1. DOI: [10.1007/978-3-319-96884-1_25](https://doi.org/10.1007/978-3-319-96884-1_25).

- [10] Dan Boneh, Benedikt Bünz, and Ben Fisch. “A Survey of Two Verifiable Delay Functions Using Proof of Exponentiation.” In: *IACR Communications in Cryptology* 1.1 (2024). ISSN: 3006-5496. DOI: [10.62056/av7tudhdj](https://doi.org/10.62056/av7tudhdj). URL: <https://cic.iacr.org/p/1/1/7> (visited on 05/24/2026).
- [11] Samira Briongos, Ghassan Karame, Claudio Soriente, and Anika Wilde. “No Forking Way: Detecting Cloning Attacks on Intel SGX Applications.” In: *Proceedings of the 39th Annual Computer Security Applications Conference*. ACSAC ’23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 744–758. ISBN: 979-8-4007-0886-2. DOI: [10.1145/3627106.3627187](https://doi.org/10.1145/3627106.3627187). URL: <https://dl.acm.org/doi/10.1145/3627106.3627187> (visited on 02/27/2026).
- [12] Jo Van Bulck, Frank Piessens, and Raoul Strackx. “SGX-Step: A Practical Attack Framework for Precise Enclave Execution Control.” In: *Proceedings of the 2nd Workshop on System Software for Trusted Execution*. SysTEX’17. New York, NY, USA: Association for Computing Machinery, 2017, pp. 1–6. ISBN: 978-1-4503-5097-6. DOI: [10.1145/3152701.3152706](https://doi.org/10.1145/3152701.3152706). URL: <https://dl.acm.org/doi/10.1145/3152701.3152706> (visited on 06/19/2026).
- [13] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. “Intel TDX Demystified: A Top-Down Approach.” In: *ACM Comput. Surv.* 56.9 (2024), 238:1–238:33. ISSN: 0360-0300. DOI: [10.1145/3652597](https://doi.org/10.1145/3652597). URL: <https://dl.acm.org/doi/10.1145/3652597> (visited on 02/27/2026).
- [14] Victor Costan and Srinivas Devadas. *Intel SGX Explained*. Publication info: Preprint. 2016. URL: <https://eprint.iacr.org/2016/086> (visited on 02/27/2026).
- [15] Bo Guan. *Designing Scalable Rate Limiting Systems: Algorithms, Architecture, and Distributed Solutions*. 2026. URL: <https://arxiv.org/abs/2602.11741v1> (visited on 05/10/2026).
- [16] André Martin, Cong Lian, Franz Gregor, Robert Krahn, Valerio Schiavoni, Pascal Felber, and Christof Fetzer. “ADAM-CS: Advanced Asynchronous Monotonic Counter Service.” In: *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2021, pp. 426–437. DOI: [10.1109/DSN48987.2021.00053](https://doi.org/10.1109/DSN48987.2021.00053). URL: <https://ieeexplore.ieee.org/document/9505080> (visited on 03/10/2026).
- [17] Sinisa Matetic, Mansoor Ahmed, Kari Kostiaainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels, and Srdjan Capkun. “ROTE: Rollback Protection for Trusted Execution.” In: *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, Aug. 2017, pp. 1289–1306. ISBN: 978-1-931971-40-9. URL: <https://www.usenix.org/conference/sec17> (visited on 05/10/2026).

- [//www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/matetic](http://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/matetic).
- [18] Jesse De Meulemeester, David Oswald, Ingrid Verbauwhede, and Jo Van Bulck. “Battering RAM: Lowcost interposer attacks on confidential computing via dynamic memory aliasing.” In: *47th IEEE Symposium on Security and Privacy (S&P)*. 2026.
 - [19] Satoshi Nakamoto. *Bitcoin: A peer-to-peer electronic cash system*. 2008. URL: <https://bitcoin.org/bitcoin.pdf>.
 - [20] Colin Percival. *Stronger key derivation via sequential memory-hard functions*. 2009. URL: https://www.bsdcan.org/2009/schedule/attachments/87_scrypt.pdf.
 - [21] Ronald L. Rivest, Adi Shamir, and David A. Wagner. *Time-lock Puzzles and Timed-release Crypto*. 1996. URL: <https://dspace.mit.edu/handle/1721.1/149822> (visited on 03/08/2026).
 - [22] J.H. Saltzer and M.D. Schroeder. “The protection of information in computer systems.” In: *Proceedings of the IEEE* 63.9 (1975), pp. 1278–1308. ISSN: 1558-2256. DOI: [10.1109/PROC.1975.9939](https://doi.org/10.1109/PROC.1975.9939). URL: <https://ieeexplore.ieee.org/abstract/document/1451869> (visited on 05/09/2026).
 - [23] Jianxiong Shao, Yu Qin, and Dengguo Feng. “Formal analysis of HMAC authorisation in the TPM2.0 specification.” In: *IET Information Security* 12.2 (2018), pp. 133–140. ISSN: 1751-8717. DOI: [10.1049/iet-ifs.2016.0005](https://doi.org/10.1049/iet-ifs.2016.0005). URL: <https://onlinelibrary.wiley.com/doi/abs/10.1049/iet-ifs.2016.0005> (visited on 03/07/2026).
 - [24] *Supporting Intel® SGX on Multisocket Platforms*. URL: <https://www.intel.com/content/www/us/en/content-details/843058/supporting-intel-sgx-on-multisocket-platforms.html> (visited on 07/01/2026).
 - [25] *TPM 2.0 Library*. URL: <https://trustedcomputinggroup.org/resource/tpm-library-specification/> (visited on 03/05/2026).
 - [26] Annika Wilde, Samira Briongos, Claudio Soriente, and Ghassan Karame. *The Real Menace of Cloning Attacks on SGX Applications*. version: 1. 2026. DOI: [10.48550/arXiv.2601.09273](https://doi.org/10.48550/arXiv.2601.09273). arXiv: [2601.09273](https://arxiv.org/abs/2601.09273). URL: <http://arxiv.org/abs/2601.09273> (visited on 02/27/2026).
 - [27] Peterson Yuhala, Christian Göttel, Jämes Ménétrey, Valerio Schiavoni, David Kozhaya, and Pascal Felber. “On Real-Time Guarantees in Intel SGX and TDX.” In: *LIPICs, Volume 335, ECRTS 2025 335* (2025), 8:1–8:25. ISSN: 1868-8969. DOI: [10.4230/LIPICs.ECRTS.2025.8](https://doi.org/10.4230/LIPICs.ECRTS.2025.8). URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPICs.ECRTS.2025.8> (visited on 03/02/2026).

- [28] *MBEDTLS/SECURITY*. URL: <https://github.com/Mbed-TLS/mbedtls/blob/development/SECURITY.md> (visited on 06/23/2026).

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Eigenständigkeitserklärung

Die unterzeichnete Eigenständigkeitserklärung ist Bestandteil jeder während des Studiums verfassten schriftlichen Arbeit. Eine der folgenden zwei Optionen ist **in Absprache mit der verantwortlichen Betreuungsperson** verbindlich auszuwählen:

- ☐ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe, namentlich, dass mir niemand beim Verfassen der Arbeit geholfen hat. Davon ausgenommen sind sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson. Es wurden keine Technologien der generativen künstlichen Intelligenz¹ verwendet.
- ☒ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe. Dabei habe ich nur die erlaubten Hilfsmittel verwendet, darunter sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson sowie Technologien der generativen künstlichen Intelligenz. Deren Einsatz und Kennzeichnung ist mit der Betreuungsperson abgesprochen.

Titel der Arbeit:

Time-Lock Puzzles for Stateless Rate-Limiting in
Trusted Execution Environments

Verfasst von:

Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich.

Name(n):

Mantel

Vorname(n):

Floritz

Ich bestätige mit meiner Unterschrift:

- Ich habe mich an die Regeln des «[Zitierleitfadens](#)» gehalten.
- Ich habe alle Methoden, Daten und Arbeitsabläufe wahrheitsgetreu und vollständig dokumentiert.
- Ich habe alle Personen erwähnt, welche die Arbeit wesentlich unterstützt haben.

Ich nehme zur Kenntnis, dass die Arbeit mit elektronischen Hilfsmitteln auf Eigenständigkeit überprüft werden kann.

Ort, Datum

Zürich, 22.06.2026

Unterschrift(en)



Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich. Durch die Unterschriften bürgen sie grundsätzlich gemeinsam für den gesamten Inhalt dieser schriftlichen Arbeit.

¹ Für weitere Informationen konsultieren Sie bitte die Webseiten der ETH Zürich, bspw. <https://ethz.ch/de/die-eth-zuerich/lehre/ai-in-education.html> und <https://library.ethz.ch/forschen-und-publizieren/Wissenschaftliches-Schreiben-an-der-ETH-Zuerich.html> (Änderungen vorbehalten).